

**DVWA Penetration Testing Final Report**  
**WEB APPLICATION PENETRATION TESTING ON DVWA WITH**  
**INCIDENT RESPONSE**

Submitted By : Bollepally Keerthana

Submitted To : ApexPlanet Software Pvt. Ltd.

Internship Program : Cybersecurity & Ethical Hacking Internship

Task Number : Task 5 – Capstone Project & Incident Response

Internship Id : APSPL2641960

Submission Date : 8-sep-2026 Technologies

Used

- Kali Linux
- DVWA (Damn Vulnerable Web Application)
- Apache Web Server
- MariaDB
- Nmap
- Nikto
- Burp Suite
- SQLMap
- Hydra
- Wireshark

# CERTIFICATE

This is to certify that **Ms. Vadla Laxmi** has successfully completed the project entitled **"Web Application Penetration Testing on DVWA with Incident Response"** as part of the **ApexPlanet Software Pvt. Ltd. Cybersecurity & Ethical Hacking Internship**, under **Task 5 – Capstone Project & Incident Response**.

The project was carried out in a controlled virtual laboratory environment using **Kali Linux** and the **Damn Vulnerable Web Application (DVWA)**. The work involved the complete penetration testing lifecycle, including project planning, reconnaissance, network scanning, vulnerability assessment, web application security testing, SQL Injection, Cross-Site Scripting (XSS), File Inclusion, Command Injection, Brute Force testing, log analysis, incident response simulation, risk assessment, mitigation recommendations, and preparation of a comprehensive penetration testing report.

This project has been completed for educational purposes by following ethical hacking principles and responsible security practices. All security assessments and exploitation activities described in this report were performed only on authorized systems within a private laboratory environment and were conducted in accordance with the objectives of the internship.

This report is submitted as partial fulfillment of the requirements for **Task 5 – Capstone Project & Incident Response** of the **ApexPlanet Cybersecurity & Ethical Hacking Internship Program**.

**Bollepally Keerthana**

## ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **ApexPlanet Software Pvt. Ltd.** for providing me with the opportunity to participate in the **Cybersecurity & Ethical Hacking Internship**. This internship has provided me with valuable practical exposure to cybersecurity concepts, ethical hacking methodologies, and penetration testing techniques through hands-on learning and real-world tasks.

I am thankful to the ApexPlanet team for designing a well-structured internship program that enabled me to strengthen my technical knowledge in web application security, network security, vulnerability assessment, incident response, and security reporting. The practical assignments helped me understand the complete penetration testing lifecycle and enhanced my problem-solving and analytical skills.

I would also like to express my appreciation to all the mentors, trainers, and support team members at ApexPlanet for their continuous guidance, encouragement, and valuable learning resources throughout the internship. Their support motivated me to complete this project successfully.

This project, "**Web Application Penetration Testing on DVWA with Incident Response**," was carried out in a controlled laboratory environment using **Kali Linux** and the **Damn Vulnerable Web Application (DVWA)** for educational purposes. The project involved reconnaissance, network scanning, vulnerability assessment, SQL Injection testing, Cross-Site Scripting (XSS), File Inclusion, Command Injection, Brute Force testing, log analysis, incident response simulation, risk assessment, and mitigation recommendations.

Finally, I would like to thank everyone who contributed directly or indirectly to the successful completion of this internship project. The knowledge and practical experience gained during this internship will be invaluable in my academic journey and future career in the field of cybersecurity.

**Bollepally Keerthana**

Cybersecurity & Ethical Hacking Intern

ApexPlanet Software Pvt. Ltd.

## Table of Contents

|                                                    |    |
|----------------------------------------------------|----|
| DVWA Penetration Testing Final Report .....        | 1  |
| Chapter 1 – Introduction.....                      | 6  |
| 1.1 Introduction .....                             | 6  |
| 1.3 Project Scope.....                             | 9  |
| 1.4 Project Goals .....                            | 10 |
| Chapter 2 – Project Planning.....                  | 11 |
| 2.1 Project Overview .....                         | 11 |
| 2.2 Project Methodology .....                      | 12 |
| 2.3 Project Timeline .....                         | 13 |
| 2.4 Lab Environment .....                          | 14 |
| 2.5 Network Diagram .....                          | 17 |
| Chapter 3 – Tools and Technologies .....           | 18 |
| 3.1 Kali Linux.....                                | 18 |
| 3.2 DVWA.....                                      | 18 |
| 3.3 Apache Web Server .....                        | 19 |
| 3.4 MariaDB .....                                  | 20 |
| 3.5 Nmap .....                                     | 21 |
| 3.6 Nikto.....                                     | 22 |
| 3.7 Burp Suite.....                                | 23 |
| 3.8 SQLMap .....                                   | 25 |
| 3.9 Hydra.....                                     | 28 |
| 3.10 Wireshark.....                                | 29 |
| Chapter 4 – Reconnaissance and Scanning .....      | 30 |
| 4.1 IP Configuration .....                         | 30 |
| Chapter 5 – Web Application Security Testing ..... | 33 |
| 5.1 Burp Suite Analysis .....                      | 33 |
| 5.2 SQL Injection .....                            | 34 |
| 5.3 SQLMap Automation .....                        | 37 |

|                                                     |    |
|-----------------------------------------------------|----|
| 5.4 Stored XSS .....                                | 39 |
| 5.5 Reflected XSS.....                              | 40 |
| 5.6 File Inclusion .....                            | 42 |
| 5.7 Command Injection .....                         | 43 |
| 5.8 Brute Force using Hydra.....                    | 46 |
| Chapter 6 – Log Analysis and Incident Response..... | 48 |
| 6.1 Apache Access Log Analysis .....                | 48 |
| 6.2 Detection.....                                  | 49 |
| 6.3 Containment.....                                | 49 |
| 6.4 Eradication.....                                | 49 |
| 6.5 Recovery.....                                   | 49 |
| 6.6 Lessons Learned .....                           | 49 |
| Chapter 7 – Risk Assessment and Mitigation .....    | 50 |
| 7.1 Risk Assessment.....                            | 50 |
| 7.2 Vulnerability Severity.....                     | 51 |
| 7.3 Mitigation Strategies.....                      | 52 |
| Chapter 8 – Results and Discussion .....            | 53 |
| 8.1 Testing Summary.....                            | 53 |
| 8.2 Findings .....                                  | 53 |
| Chapter 9 – Conclusion and Future Scope .....       | 54 |
| 9.1 Conclusion.....                                 | 54 |
| 9.2 Future Scope.....                               | 54 |
| References .....                                    | 55 |

# Chapter 1 – Introduction

## 1.1 Introduction

In today's digital world, web applications play a crucial role in providing online services for businesses, educational institutions, government organizations, and individuals. As the number of web applications continues to grow, so do the security threats targeting them. Cyber attackers constantly attempt to exploit vulnerabilities in web applications to gain unauthorized access, steal sensitive information, manipulate data, or disrupt services. Therefore, ensuring the security of web applications has become an essential aspect of cybersecurity.

Web Application Penetration Testing is a systematic process of evaluating the security of a web application by identifying, analyzing, and safely exploiting vulnerabilities in a controlled environment. It helps security professionals understand how attackers can compromise an application and provides recommendations to eliminate identified security weaknesses. Penetration testing not only improves the security posture of an application but also helps organizations comply with industry security standards and best practices.

This project focuses on performing a comprehensive penetration test on the **Damn Vulnerable Web Application (DVWA)** using **Kali Linux**. DVWA is an intentionally vulnerable web application designed specifically for learning web application security, ethical hacking, and penetration testing techniques in a safe and controlled environment. It provides various vulnerable modules that simulate real-world web security issues, allowing learners to gain practical experience without affecting production systems.

The project was carried out as part of the **ApexPlanet Software Pvt. Ltd. Cybersecurity & Ethical Hacking Internship**, under **Task 5 – Capstone Project & Incident Response**. The primary objective of this project is to perform a complete penetration testing assessment of DVWA by following a standard penetration testing methodology, including project planning, reconnaissance, network scanning, vulnerability assessment, exploitation, log analysis, incident response simulation, risk assessment, and mitigation recommendations.

During this assessment, several common web application vulnerabilities based on the **OWASP Top 10** security risks were examined. These include **SQL Injection (SQLi)**, **Cross-Site Scripting (Stored and Reflected XSS)**, **File Inclusion**, **Command Injection**, and **Brute Force Authentication**. Industry-standard cybersecurity tools such

as **Nmap**, **Nikto**, **Burp Suite**, **SQLMap**, **Hydra**, and **Wireshark** were used to perform security testing, analyze network traffic, identify vulnerabilities, and collect evidence for reporting purposes.

## 1.2 Project Objectives

The primary objective of this project is to perform a comprehensive Web Application Penetration Test on the Damn Vulnerable Web Application (DVWA) using Kali Linux in a secure and controlled laboratory environment. The project aims to identify, analyze, and demonstrate common web application vulnerabilities while following ethical hacking principles and industry-standard penetration testing methodologies. It provides practical experience in assessing web application security, understanding potential cyber threats, and implementing appropriate mitigation techniques.

The specific objectives of this project are:

- To establish a secure penetration testing laboratory using Kali Linux and DVWA for conducting ethical security assessments.
- To perform reconnaissance and network scanning using Nmap to identify the target system, open ports, running services, and operating system details.
- To assess the security of the web server using Nikto by identifying common vulnerabilities and security misconfigurations.
- To intercept, inspect, and analyze HTTP requests and responses using Burp Suite for understanding web application communication.
- To identify and demonstrate common OWASP Top 10 web application vulnerabilities, including SQL Injection (SQLi), Cross-Site Scripting (Stored and Reflected XSS), File Inclusion, Command Injection, and Brute Force Authentication.
- To automate SQL Injection testing using SQLMap and evaluate the information that can be retrieved from the vulnerable application.
- To perform password auditing using Hydra against the DVWA authentication module in a controlled environment.
- To monitor and analyze Apache server logs for detecting and understanding attack activities during penetration testing.
- To simulate an Incident Response process by demonstrating the stages of detection, containment, eradication, recovery, and documentation of security incidents.
- To evaluate the severity and potential impact of identified vulnerabilities through a structured Risk Assessment and recommend suitable mitigation techniques based on secure coding practices and industry standards.



- To prepare a comprehensive penetration testing report that includes the testing methodology, tools used, commands executed, findings, screenshots, risk assessment, mitigation strategies, and conclusions.
- To strengthen practical knowledge of ethical hacking, web application security, vulnerability assessment, incident response, and cybersecurity reporting, while gaining hands-on experience in conducting security assessments within an authorized laboratory environment.

### 1.3 Project Scope

This project focuses on performing a comprehensive **Web Application Penetration Test** on the **Damn Vulnerable Web Application (DVWA)** using **Kali Linux** in a secure and isolated laboratory environment. The scope of this project is limited to authorized testing within the virtual lab and is intended solely for educational and learning purposes. No external systems, websites, or unauthorized networks are included in the assessment.

The penetration testing process covers the complete security assessment lifecycle, beginning with **project planning**, followed by **reconnaissance**, **network scanning**, **service identification**, **vulnerability assessment**, **web application security testing**, **log analysis**, **incident response simulation**, **risk assessment**, and **mitigation recommendations**. Standard cybersecurity tools such as **Nmap**, **Nikto**, **Burp Suite**, **SQLMap**, **Hydra**, and **Wireshark** are used to perform the assessment and collect evidence.

The security assessment specifically evaluates several common web application vulnerabilities based on the **OWASP Top 10** security risks. These include **SQL Injection (SQLi)**, **Stored and Reflected Cross-Site Scripting (XSS)**, **File Inclusion**, **Command Injection**, and **Brute Force Authentication**. Each vulnerability is identified, tested in a controlled manner, analyzed for its potential impact, and accompanied by appropriate mitigation strategies based on industry best practices.

In addition to vulnerability testing, the project includes monitoring **Apache web server logs** to observe attack activities and simulate an **Incident Response** process involving detection, containment, eradication, recovery, and documentation of security incidents. The findings are documented in a professional penetration testing report with supporting screenshots, commands, observations, and recommendations.

This project **does not include** testing of public websites, cloud infrastructure, wireless networks, social engineering attacks, denial-of-service attacks, malware development, or any activity that may affect systems outside the authorized laboratory environment. All

testing activities are conducted responsibly and ethically in accordance with cybersecurity best practices.

The overall scope of this project is to provide practical experience in identifying and understanding web application vulnerabilities, applying ethical hacking techniques in a safe environment, and developing professional skills in penetration testing, security assessment, incident response, and technical reporting. The knowledge gained through this project helps build a strong foundation for real-world cybersecurity and secure application development.

## 1.4 Project Goals

The goal of this project is to gain practical knowledge and hands-on experience in **web application penetration testing** by assessing the security of the **Damn Vulnerable Web Application (DVWA)** within a controlled laboratory environment. This project is designed to strengthen the understanding of cybersecurity concepts, ethical hacking methodologies, and industry-standard penetration testing practices while following responsible and authorized security testing procedures.

The project aims to develop practical skills in identifying, analyzing, and exploiting common web application vulnerabilities such as **SQL Injection (SQLi)**, **Cross-Site Scripting (Stored and Reflected XSS)**, **File Inclusion**, **Command Injection**, and **Brute Force Authentication**. It also focuses on using professional cybersecurity tools including **Nmap**, **Nikto**, **Burp Suite**, **SQLMap**, **Hydra**, and **Wireshark** to perform reconnaissance, vulnerability assessment, web application testing, and network analysis.

Another important goal is to understand the complete penetration testing lifecycle, including project planning, reconnaissance, scanning, exploitation, documentation, risk assessment, mitigation, and incident response. The project also aims to improve the ability to analyze security findings, monitor server logs, identify attack indicators, and recommend effective security controls to reduce potential risks.

Furthermore, this project is intended to enhance technical documentation and reporting skills by preparing a professional penetration testing report containing testing methodologies, commands executed, screenshots, observations, findings, risk analysis, mitigation strategies, and conclusions. Overall, the project serves as a practical learning experience that strengthens problem-solving abilities, ethical hacking skills, and cybersecurity knowledge while preparing for real-world roles in information security and penetration testing.

## Chapter 2 – Project Planning

### 2.1 Project Overview

This project, "**Web Application Penetration Testing on DVWA with Incident Response,**" was carried out as part of the **ApexPlanet Software Pvt. Ltd. Cybersecurity & Ethical Hacking Internship – Task 5 (Capstone Project & Incident Response)**. The project focuses on performing a comprehensive security assessment of the **Damn Vulnerable Web Application (DVWA)** using **Kali Linux** in a secure and controlled laboratory environment. The purpose of this project is to simulate a real-world penetration testing engagement while following ethical hacking principles and authorized security testing practices.

The project follows a structured penetration testing methodology beginning with project planning and lab setup, followed by reconnaissance, network scanning, service enumeration, vulnerability assessment, web application testing, incident response simulation, and final reporting. Industry-standard cybersecurity tools such as **Nmap**, **Nikto**, **Burp Suite**, **SQLMap**, **Hydra**, and **Wireshark** are used throughout the assessment to identify vulnerabilities, analyze network traffic, intercept HTTP requests, and document security findings.

During the assessment, several common web application vulnerabilities based on the **OWASP Top 10** security risks are evaluated, including **SQL Injection (SQLi)**, **Stored and Reflected Cross-Site Scripting (XSS)**, **File Inclusion**, **Command Injection**, and **Brute Force Authentication**. Each vulnerability is tested within the DVWA environment to understand its impact, demonstrate controlled exploitation, and recommend appropriate mitigation techniques to improve application security.

In addition to vulnerability testing, the project includes monitoring Apache web server logs and simulating an incident response process involving detection, containment, eradication, recovery, and lessons learned. All findings are documented in a professional penetration testing report containing commands executed, screenshots, observations, risk assessment, mitigation strategies, and conclusions. This project provides practical experience in ethical hacking, vulnerability assessment, incident response, and technical reporting while reinforcing the importance of secure software development and responsible cybersecurity practices.

## 2.2 Project Methodology

This project follows a structured and systematic **penetration testing methodology** to evaluate the security of the **Damn Vulnerable Web Application (DVWA)** in a controlled laboratory environment. The methodology is designed to simulate a real-world web application security assessment while ensuring that all testing activities are conducted ethically and only on authorized systems. Each phase of the methodology contributes to identifying vulnerabilities, assessing their impact, and recommending effective mitigation strategies.

The first phase of the project involves **Project Planning**, where the objectives, scope, tools, and testing approach are defined before beginning the assessment. A secure virtual lab environment consisting of a **Kali Linux attacker machine** and a **DVWA target application** is prepared to ensure safe and isolated testing.

The second phase is **Reconnaissance**, which focuses on gathering information about the target system. Basic network connectivity is verified, and the target IP address is identified before performing further security assessments.

The third phase is **Scanning and Enumeration**, where tools such as **Nmap** and **Nikto** are used to identify open ports, running services, operating system details, and potential web server vulnerabilities. This phase provides valuable information about the target system that helps in planning the exploitation process.

The fourth phase is **Web Application Security Testing**, during which multiple vulnerabilities are assessed using both manual and automated techniques. **Burp Suite** is used to intercept and analyze HTTP requests, while **SQLMap** is used to automate SQL Injection testing. Additional testing is performed to identify vulnerabilities such as **Stored and Reflected Cross-Site Scripting (XSS)**, **File Inclusion**, **Command Injection**, and **Brute Force Authentication** within the DVWA environment.

The fifth phase involves **Log Analysis and Incident Response**, where Apache server logs are monitored to observe attack activities and understand how security events can be detected. A simulated incident response process is then carried out by demonstrating the stages of **detection, containment, eradication, recovery, and lessons learned** after identifying security incidents.

The final phase is **Documentation and Reporting**, where all findings, screenshots, commands, observations, vulnerability assessments, risk analysis, and mitigation recommendations are compiled into a professional penetration testing report. This methodology provides a structured approach to assessing web application security while strengthening practical knowledge of ethical hacking, vulnerability assessment, incident response, and cybersecurity reporting.

## 2.3 Project Timeline

The project was completed by following a structured penetration testing approach in a controlled laboratory environment. Each phase was carried out sequentially to ensure systematic assessment, proper documentation, and effective analysis of the identified vulnerabilities. The timeline began with project planning and environment setup, followed by reconnaissance, scanning, vulnerability assessment, exploitation, incident response simulation, and final report preparation.

| Phase    | Activity                     | Description                                                                                                                                                     |
|----------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Phase 1  | Project Planning             | Defined project objectives, scope, methodology, tools, and prepared the project documentation.                                                                  |
| Phase 2  | Lab Environment Setup        | Configured Kali Linux, DVWA, Apache Web Server, MariaDB, and verified network connectivity.                                                                     |
| Phase 3  | Reconnaissance               | Identified the target system, verified connectivity, and collected basic information using network commands.                                                    |
| Phase 4  | Network Scanning             | Performed port scanning, service detection, operating system detection, and web server scanning using Nmap and Nikto.                                           |
| Phase 5  | Vulnerability Assessment     | Analyzed the target application using Burp Suite and identified potential security vulnerabilities.                                                             |
| Phase 6  | Web Application Testing      | Performed SQL Injection, Cross-Site Scripting (Stored & Reflected XSS), File Inclusion, Command Injection, and Brute Force testing within the DVWA environment. |
| Phase 7  | Log Analysis                 | Monitored Apache access logs to observe attack activities and collect evidence of penetration testing.                                                          |
| Phase 8  | Incident Response Simulation | Simulated the incident response process, including detection, containment, eradication, recovery, and lessons learned.                                          |
| Phase 9  | Risk Assessment & Mitigation | Evaluated the severity of identified vulnerabilities and recommended appropriate mitigation strategies.                                                         |
| Phase 10 | Documentation & Reporting    | Compiled the complete penetration testing report with methodology, findings, screenshots, risk assessment, mitigation techniques, and conclusions.              |

## 2.4 Lab Environment

The penetration testing activities for this project were conducted in a **secure and isolated virtual laboratory environment** to ensure that all security assessments were performed safely, ethically, and without affecting any external systems or production networks. The laboratory environment was designed to simulate a real-world web application security testing scenario, allowing practical implementation of penetration testing techniques within an authorized setup.

The laboratory consisted of two primary components: the **attacker machine** and the **target system**. The attacker machine was configured with **Kali Linux**, a widely used penetration testing operating system that provides a comprehensive collection of cybersecurity tools required for vulnerability assessment and ethical hacking. The target system hosted the **Damn Vulnerable Web Application (DVWA)** on an **Apache Web Server** with a **MariaDB** database, providing an intentionally vulnerable platform for security testing.

The attacker and target systems were connected through a **Host-Only Network Adapter**, creating a private network that allowed communication only between the virtual machines. This configuration ensured that all penetration testing activities remained isolated from the Internet and prevented accidental interaction with external systems. Before beginning the security assessment, network connectivity between the attacker and target machines was verified using basic network diagnostic commands.

The following software and tools were installed and configured within the laboratory environment:

| Component                | Description                            |
|--------------------------|----------------------------------------|
| Operating System         | Kali Linux                             |
| Target Application       | Damn Vulnerable Web Application (DVWA) |
| Web Server               | Apache HTTP Server                     |
| Database Server          | MariaDB                                |
| Programming Language PHP |                                        |
| Web Browser              | Mozilla Firefox                        |

| <b>Component</b>           | <b>Description</b> |
|----------------------------|--------------------|
| <b>Network Type</b>        | Host-Only Adapter  |
| <b>Attacker IP Address</b> | 192.168.56.101     |
| <b>Target IP Address</b>   | 192.168.56.102     |

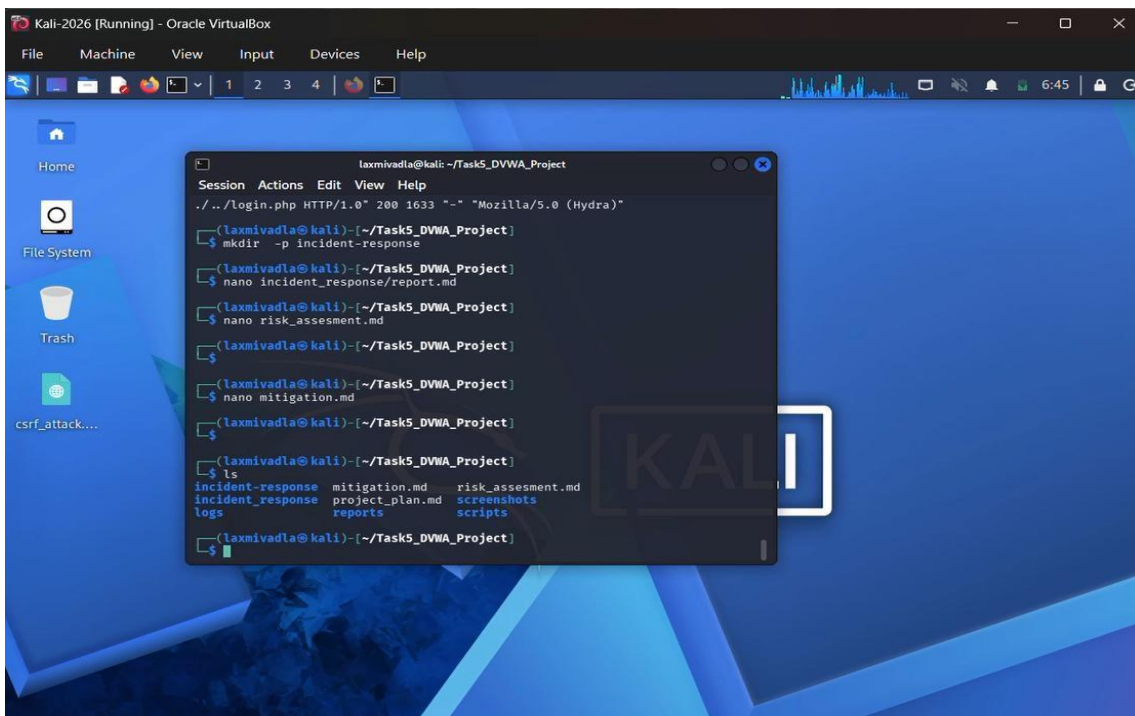
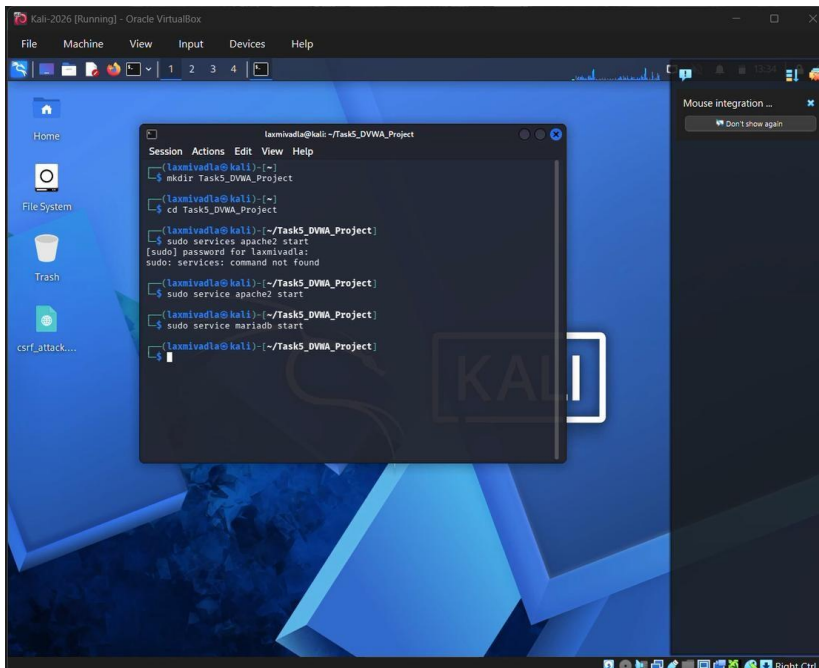
The following penetration testing tools were used during the assessment:

| <b>Tool</b>       | <b>Purpose</b>                                                                   |
|-------------------|----------------------------------------------------------------------------------|
| <b>Nmap</b>       | Host discovery, port scanning, service detection, and operating system detection |
| <b>Nikto</b>      | Web server vulnerability scanning and security misconfiguration analysis         |
| <b>Burp Suite</b> | HTTP request interception, request modification, and web application testing     |
| <b>SQLMap</b>     | Automated SQL Injection detection and database enumeration                       |
| <b>Hydra</b>      | Password auditing and brute-force authentication testing                         |
| <b>Wireshark</b>  | Network packet capture and traffic analysis                                      |

Before starting the penetration testing process, essential services including the **Apache Web Server** and **MariaDB Database Server** were started and verified to ensure that the DVWA application was functioning correctly. The application was accessed through the Mozilla Firefox browser, and the DVWA security level was configured to **Low** to facilitate learning and demonstration of common web application vulnerabilities.

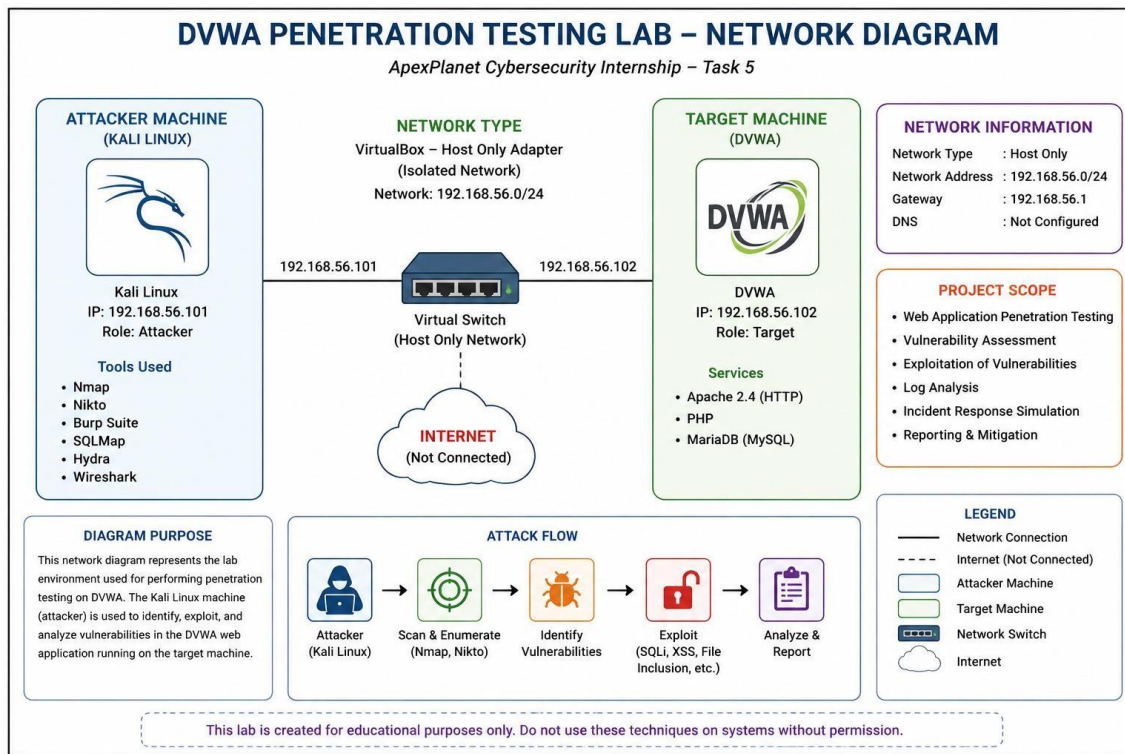
The laboratory environment provided a safe platform for performing reconnaissance, network scanning, vulnerability assessment, exploitation, log analysis, and incident response simulation. Since DVWA is intentionally designed for educational purposes, all testing activities were conducted responsibly and only within the authorized environment.

Overall, the laboratory environment played a critical role in successfully completing this project by providing a realistic yet secure platform for learning web application penetration testing techniques, understanding cybersecurity concepts, and gaining practical experience with industry-standard security tools.





## 2.5 Network Diagram



## Chapter 3 – Tools and Technologies

### 3.1 Kali Linux

**Kali Linux** is a Linux-based operating system specially designed for **cybersecurity, ethical hacking, and penetration testing**. It includes many built-in security tools that help identify and test vulnerabilities in computer systems and web applications.

In this project, Kali Linux was used as the **attacker machine** to perform security testing on the **Damn Vulnerable Web Application (DVWA)**. It was used to scan the target, identify vulnerabilities, analyze HTTP requests, and generate reports.

#### Commands Used

| Command                           | Purpose                                          |
|-----------------------------------|--------------------------------------------------|
| ip a                              | Display the IP address of Kali Linux.            |
| ping 192.168.56.102               | Check connectivity with the DVWA target machine. |
| pwd                               | Show the current working directory.              |
| ls                                | List files and folders.                          |
| cd Task5_DVWA_Project             | Move to the project folder.                      |
| mkdir -p reports screenshots logs | Create folders for project files.                |
| sudo service apache2 start        | Start the Apache web server.                     |
| sudo service mariadb start        | Start the MariaDB database server.               |

#### Role in the Project:

Kali Linux was the main operating system used throughout the project for reconnaissance, scanning, vulnerability testing, log analysis, and report preparation.

### 3.2 DVWA

**Damn Vulnerable Web Application (DVWA)** is an intentionally vulnerable web application designed for learning **web application security, ethical hacking, and penetration testing**. It provides a safe environment to practice identifying and exploiting common web vulnerabilities without affecting real systems.

In this project, DVWA was used as the **target application** to perform various security tests, including **SQL Injection, Cross-Site Scripting (XSS), File Inclusion, Command Injection, and Brute Force Authentication**. The application was hosted on the **Apache**

**Web Server** with a **MariaDB** database and accessed through the Mozilla Firefox browser.

### DVWA Modules Tested

| Module            | Purpose                                      |
|-------------------|----------------------------------------------|
| SQL Injection     | Test database security vulnerabilities.      |
| Stored XSS        | Test stored cross-site scripting attacks.    |
| Reflected XSS     | Test reflected cross-site scripting attacks. |
| File Inclusion    | Test unauthorized file access.               |
| Command Injection | Test execution of system commands.           |
| Brute Force       | Test login security using password attacks.  |

### Role in the Project

DVWA served as the **target application** throughout the penetration testing process. It allowed safe testing of common web vulnerabilities and helped understand their impact and mitigation techniques in a controlled laboratory environment.

## 3.3 Apache Web Server

### 2.4.3 Apache Web Server

**Apache Web Server** is an open-source web server used to host websites and web applications. It processes HTTP requests from users and delivers web pages through a web browser.

In this project, Apache Web Server was used to host the **Damn Vulnerable Web Application (DVWA)**, allowing the application to be accessed locally through the browser. It also generated server logs that were analyzed during the penetration testing process.

### Commands Used

| Command                                 | Purpose                      |
|-----------------------------------------|------------------------------|
| <code>sudo service apache2 start</code> | Start the Apache web server. |

| Command                                               | Purpose                                      |
|-------------------------------------------------------|----------------------------------------------|
| <code>sudo service apache2 status</code>              | Check whether the Apache service is running. |
| <code>sudo service apache2 stop</code>                | Stop the Apache web server.                  |
| <code>sudo tail -f /var/log/apache2/access.log</code> | Monitor Apache access logs in real time.     |

### Role in the Project

Apache Web Server hosted the DVWA application and processed all HTTP requests during the penetration testing. It also recorded user activities in the **access.log** file, which was used for log analysis and incident response simulation.

### Screenshot to Include:

- Apache service status (`sudo service apache2 status`)
- Apache access log (`sudo tail -f /var/log/apache2/access.log`)

## 3.4 MariaDB

### 2.4.4 MariaDB

**MariaDB** is an open-source **Database Management System (DBMS)** used to store and manage application data. In this project, MariaDB was used as the backend database for the **Damn Vulnerable Web Application (DVWA)**. It stored user information, login credentials, and other application data required for security testing.

### Commands Used

| Command                                  | Purpose                                       |
|------------------------------------------|-----------------------------------------------|
| <code>sudo service mariadb start</code>  | Start the MariaDB database service.           |
| <code>sudo service mariadb status</code> | Check whether the MariaDB service is running. |
| <code>sudo mysql</code>                  | Open the MariaDB command-line interface.      |
| <code>SHOW DATABASES;</code>             | Display all available databases.              |

### Role in the Project

MariaDB stored the DVWA database and was used during **SQL Injection testing** to demonstrate how attackers can access or manipulate database information if proper security measures are not implemented.

**Screenshot to Include:**

- MariaDB service status (sudo service mariadb status)
- Output of SHOW DATABASES; showing the **dvwa** database

### 3.5 Nmap

#### 2.4.5 Nmap

**Nmap (Network Mapper)** is an open-source network scanning tool used to discover hosts, identify open ports, detect running services, and determine the operating system of a target machine. It is one of the most widely used tools in penetration testing.

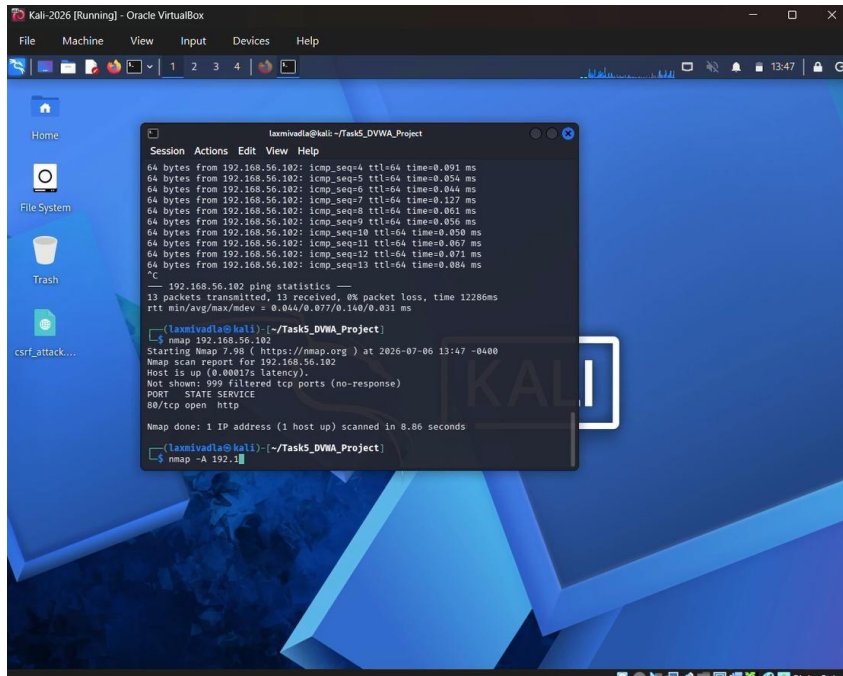
In this project, Nmap was used to scan the DVWA target machine, identify open ports, detect the Apache web server, and gather information before performing web application security testing.

**Commands Used**

| Command                     | Purpose                                                         |
|-----------------------------|-----------------------------------------------------------------|
| nmap 192.168.56.102         | Perform a basic port scan.                                      |
| nmap -sV 192.168.56.102     | Detect service versions running on open ports.                  |
| sudo nmap -O 192.168.56.102 | Detect the operating system of the target.                      |
| nmap -A 192.168.56.102      | Perform an aggressive scan (OS, services, scripts, traceroute). |

**Role in the Project**

Nmap was used to gather information about the target system before vulnerability testing. The scan identified the running web service, open ports, and system details, which helped in planning the penetration testing process.



## nmapScan

### 3.6 Nikto

#### 2.4.6 Nikto

**Nikto** is an open-source web server scanning tool used to identify common security vulnerabilities, outdated software, and configuration issues in web servers. It helps security testers find potential weaknesses before exploitation.

In this project, **Nikto** was used to scan the DVWA web server and identify possible security issues related to the Apache server configuration.

#### Command Used

##### Command

```
nikto -h
http://192.168.56.102
```

##### Purpose

Scan the target web server for vulnerabilities and security misconfigurations.

#### Role in the Project

Nikto was used to gather information about the target web server and identify potential security weaknesses. The scan results helped in understanding the server configuration before performing further penetration testing.

### Screenshot to Include:

- Nikto scan output showing the identified web server information and scan results.

NiktoScan

## 3.7 Burp Suite

**Burp Suite** is a web application security testing tool used to intercept, inspect, and modify HTTP requests and responses between a web browser and a web server. It helps security testers identify vulnerabilities in web applications.

In this project, **Burp Suite** was used to capture login requests, analyze HTTP traffic, and inspect cookies and request parameters while testing the DVWA application.

### Command Used

### Command Purpose

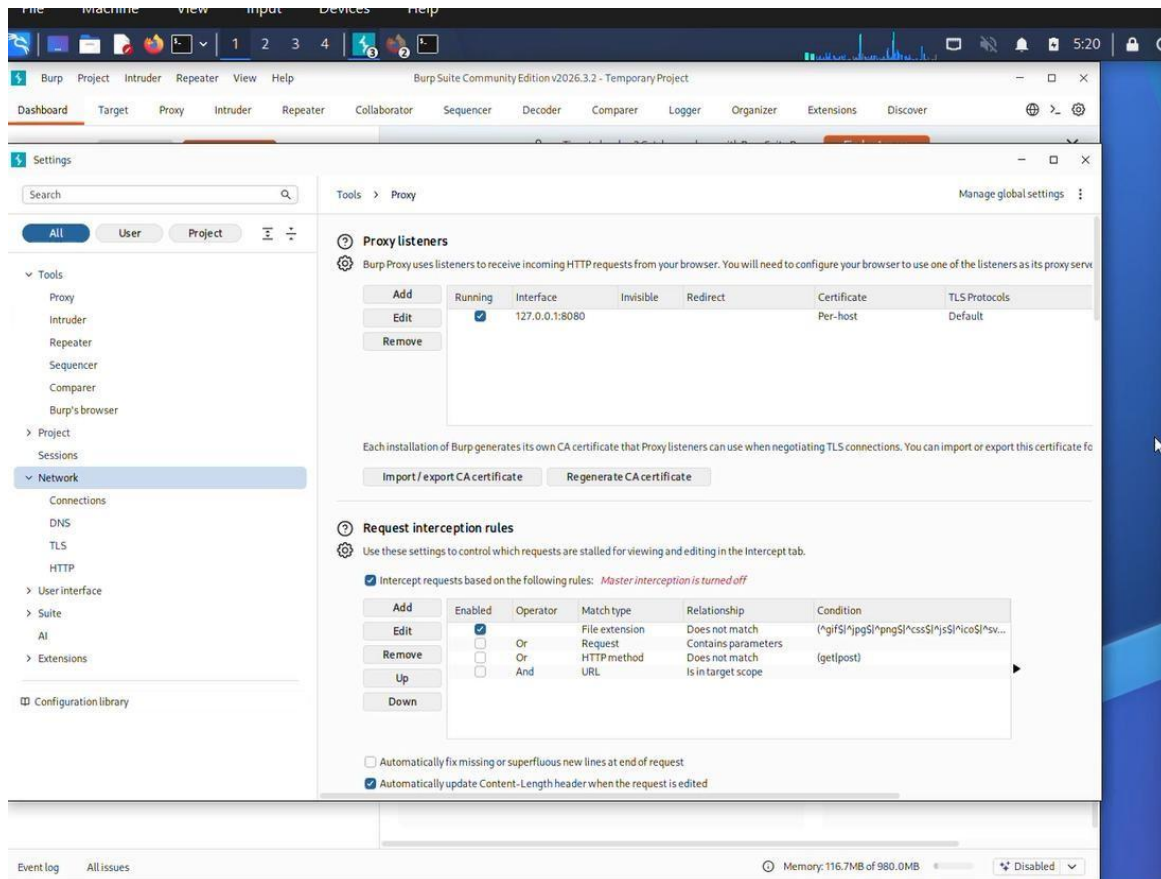
## Command Purpose

`burpsuite` Launch the Burp Suite application.

## Role in the Project

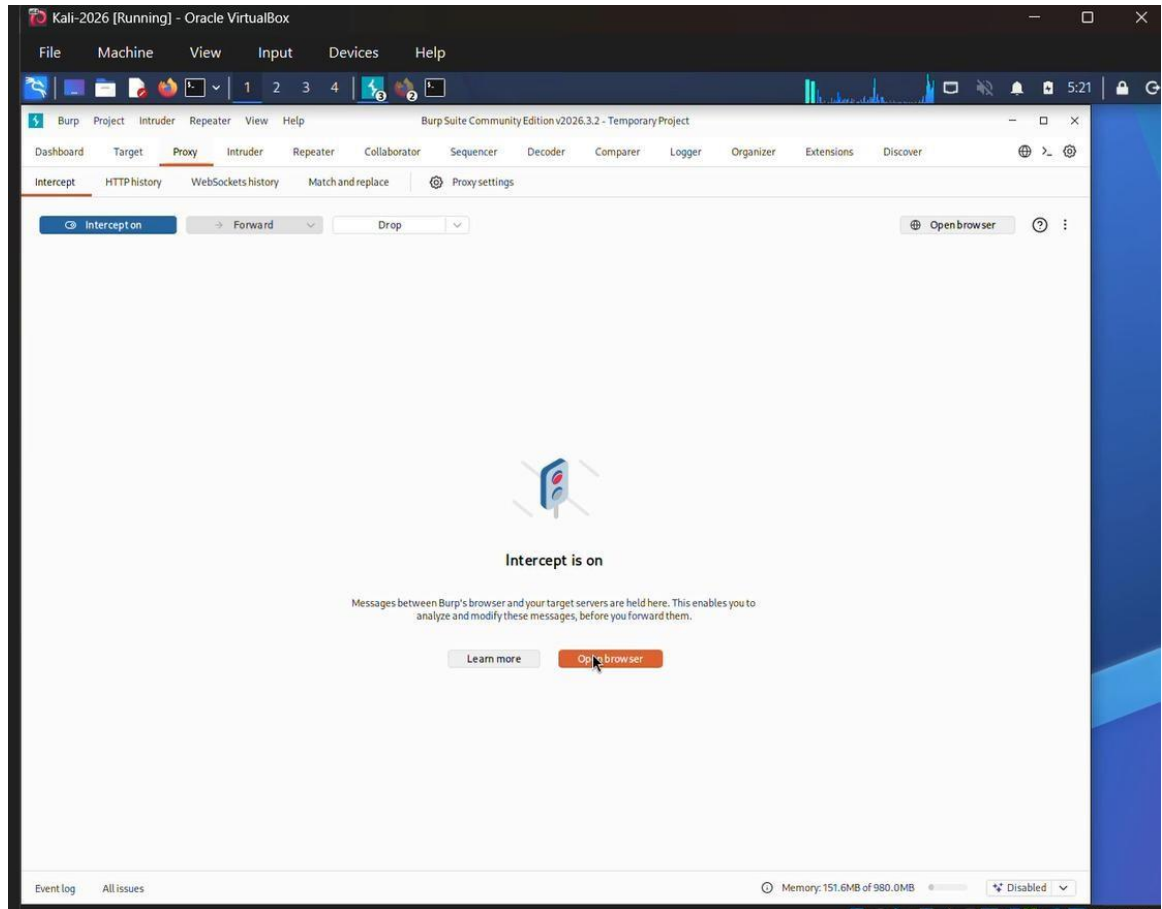
Burp Suite was used to intercept HTTP requests between the browser and DVWA, allowing the analysis of request parameters, cookies, and responses before performing web application security testing.

## Screenshot to Include:



Burp Suite Home Screen





Proxy Intercept ON showing the captured request

### 3.8 SQLMap

#### 2.4.8 SQLMap

**SQLMap** is an open-source penetration testing tool used to automatically detect and exploit **SQL Injection** vulnerabilities in web applications. It helps security testers identify databases, tables, and data that may be exposed due to insecure SQL queries.

In this project, **SQLMap** was used to test the SQL Injection vulnerability in the DVWA application and demonstrate how an attacker could retrieve database information in a controlled environment.

#### Commands Used

##### Command

##### Purpose

## Command

```
sqlmap --version
```

```
sqlmap -u "<Target_URL>" --cookie="PHPSESSID=...;  
security=low" --dbs
```

```
sqlmap -u "<Target_URL>" --cookie="PHPSESSID=...;  
security=low" -D dvwa --tables
```

```
sqlmap -u "<Target_URL>" --cookie="PHPSESSID=...;  
security=low" -D dvwa -T users --dump
```

## Purpose

Check whether  
SQLMap is installed.

Identify available  
databases.

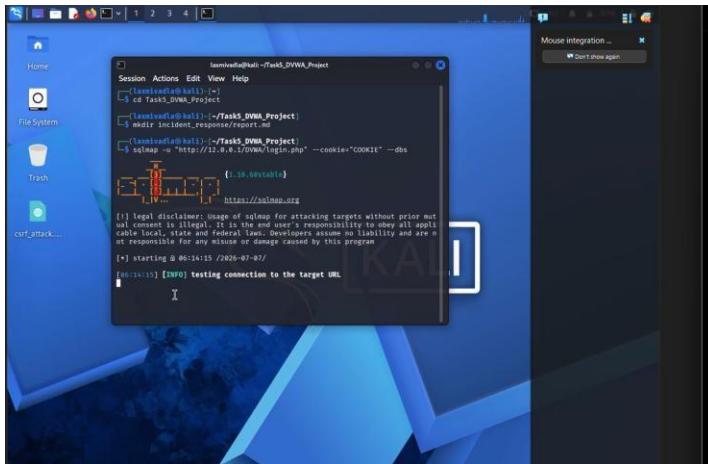
List tables in the  
DVWA database.

Retrieve data from the  
**users** table.

## Role in the Project

SQLMap was used to automate SQL Injection testing, identify the DVWA database, list database tables, and retrieve data from the vulnerable application. This demonstrated the importance of secure database queries and input validation.

## Screenshot to Include:



SQLMap detecting databases (--dbs)

```
laxmivadla@kali: ~/Task5_DVWA_Project
Session Actions Edit View Help
Payload: id=1' AND (SELECT 6860 FROM (SELECT(SLEEP(5)))geor) AND 'LWIs'='
LWIs6Submit=Submit

Type: UNION query
Title: Generic UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT(0x71786b7871,0x6a764757466b6142566
b726d5a7a494c5a424867454975465043616361647a73454578466f6a414e,0x716b767071),N
ULL-- -8Submit=Submit

[06:18:50] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.68
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[06:18:50] [INFO] fetching database names
available databases [2]:
[*] dwwa
[*] information_schema

[06:18:50] [INFO] fetched data logged to text files under '/home/laxmivadla/.
local/share/sqlmap/output/localhost'

[*] ending @ 06:18:50 /2026-07-07/

(laxmivadla@kali)~/Task5_DVWA_Project
$ sqlmap -u "http://localhost/DVWA/vulnerabilities/sql4/?id=16Submit=Submit
" --cookie="PHPSESSID=530e34688b552dc918cf4731c396edf; security=low" -D dwwa
--tables
```

SQLMap listing tables (--tables)

```
laxmivadla@kali: ~/Task5_DVWA_Project
Session Actions Edit View Help
[06:19:28] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:

Parameter: id (GET)
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 6860 FROM (SELECT(SLEEP(5)))geor) AND 'LWIs'='
LWIs6Submit=Submit

Type: UNION query
Title: Generic UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT(0x71786b7871,0x6a764757466b6142566
b726d5a7a494c5a424867454975465043616361647a73454578466f6a414e,0x716b767071),N
ULL-- -6Submit=Submit

[06:19:28] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.68
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[06:19:28] [INFO] fetching columns for table 'users' in database 'dvwa'
[06:19:28] [WARNING] reflective value(s) found and filtering out
[06:19:28] [INFO] fetching entries for table 'users' in database 'dvwa'
[06:19:28] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processi
ng with other tools [y/N] Y
[06:19:31] [INFO] writing hashes to a temporary file '/tmp/sqlmapznz0d7xq6189
/sqlmaphashes-yvf_49es.txt'
do you want to crack them via a dictionary-based attack? [Y/n/q]
```

SQLMap dumping the users table (--dump)

### 3.9 Hydra

**Hydra** is an open-source password auditing and brute-force testing tool used to evaluate the strength of login authentication. It automatically tests multiple username and password combinations to identify weak credentials.

In this project, **Hydra** was used to perform a **Brute Force Authentication** test on the DVWA login page within the controlled laboratory environment. This demonstrated how weak passwords can be exploited if proper security measures are not implemented.

#### Command Used

| Command                                                                                                                                                                                                                   | Purpose                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <code>hydra -l admin -P /usr/share/wordlists/rockyou.txt localhost http-post-form the "/dvwa/vulnerabilities/brute/:username=^USER^&amp;password=^PASS^&amp;Login=DVWA Login:F=Username and/or password incorrect"</code> | Perform a brute-force password attack on the login page using the <b>rockyou.txt</b> wordlist. |

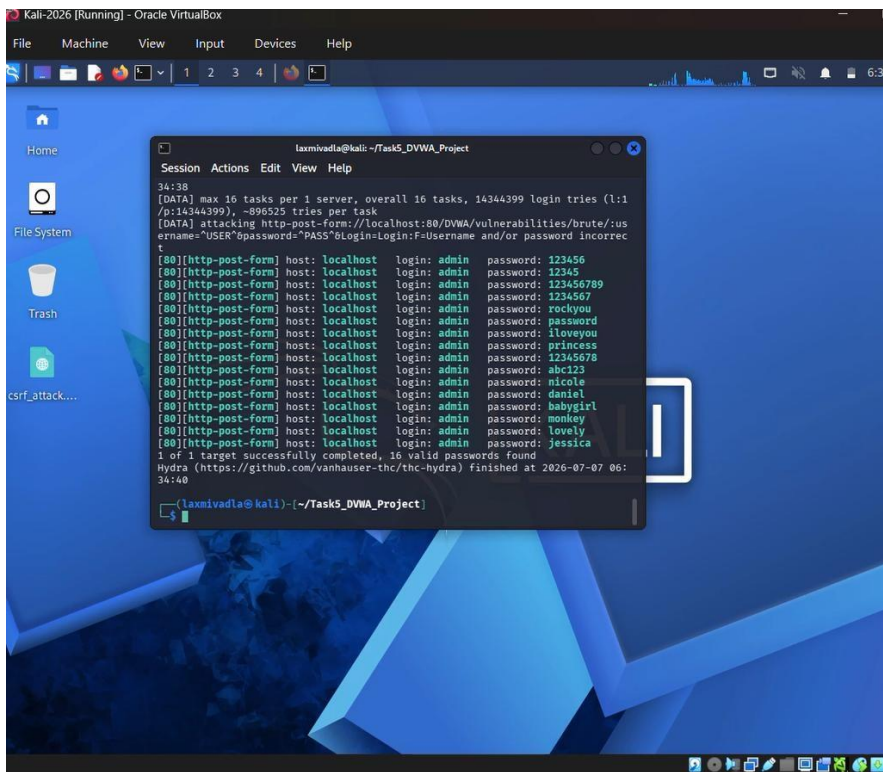
#### Role in the Project

Hydra was used to demonstrate the risks of weak passwords by performing a controlled brute-force attack against the DVWA application. This highlighted the importance of strong passwords, account lockout policies, and multi-factor authentication.

#### Screenshot to Include:

- Hydra command execution
- Hydra output showing the testing process (and results, if applicable)

**Note:** This test was performed **only on the DVWA application** within the authorized lab environment for educational purposes.



### 3.10 Wireshark

**Wireshark** is an open-source network protocol analyzer used to capture and analyze network traffic. It helps security professionals monitor data packets, identify communication between systems, and troubleshoot network-related issues.

In this project, **Wireshark** was used to capture and analyze the network traffic generated during penetration testing. It helped observe HTTP communication between the **Kali Linux** attacker machine and the **DVWA** web application.

#### Command Used

#### Command Purpose

wireshark Launch the Wireshark application.

#### Role in the Project

Wireshark was used to capture network packets and analyze the communication between the attacker and the target system. It helped understand how HTTP requests and responses are transmitted during penetration testing and provided evidence for traffic analysis.

## Chapter 4 – Reconnaissance and Scanning

### 4.1 IP Configuration

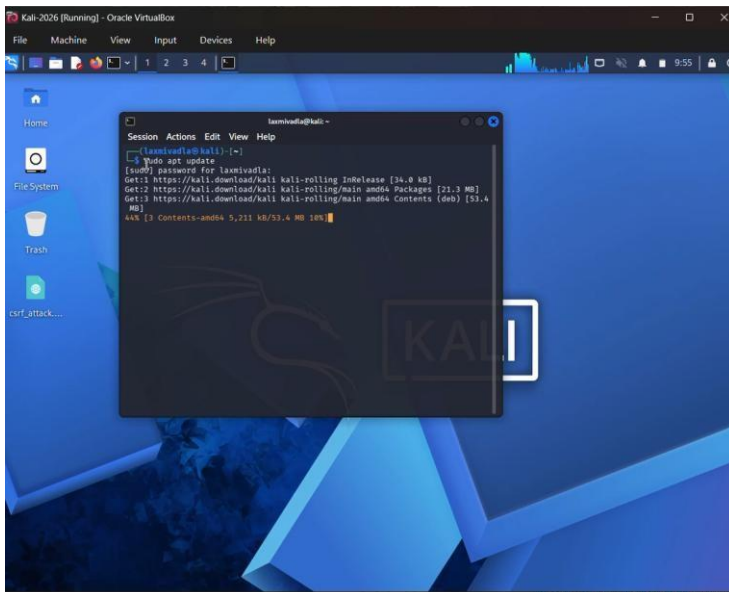
| S.No | Command                                                                  | Description                                                                            |
|------|--------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| 1    | <code>ip a</code>                                                        | Displays the IP address and network interface details of the Kali Linux machine.       |
| 2    | <code>ping 192.168.56.102</code>                                         | Verifies network connectivity between the attacker (Kali Linux) and the target (DVWA). |
| 3    | <code>pwd</code>                                                         | Displays the current working directory in the terminal.                                |
| 4    | <code>ls</code>                                                          | Lists the files and directories in the current location.                               |
| 5    | <code>cd Task5_DVWA_Project</code>                                       | Changes the current directory to the project folder.                                   |
| 6    | <code>mkdir -p reports screenshots logs incident_response scripts</code> | Creates folders to organize reports, screenshots, logs, and other project files.       |
| 7    | <code>sudo service apache2 status</code>                                 | Checks whether the Apache Web Server is running.                                       |
| 8    | <code>sudo service apache2 start</code>                                  | Starts the Apache Web Server if it is stopped.                                         |
| 9    | <code>sudo service mariadb status</code>                                 | Checks whether the MariaDB database service is running.                                |
| 10   | <code>sudo service mariadb start</code>                                  | Starts the MariaDB database service.                                                   |
| 11   | <code>nmap 192.168.56.102</code>                                         | Performs a basic scan to identify open ports on the target system.                     |
| 12   | <code>nmap -sV 192.168.56.102</code>                                     | Detects the versions of services running on the target machine.                        |
| 13   | <code>sudo nmap -O 192.168.56.102</code>                                 | Detects the operating system running on the                                            |

## S.No Command

## Description

target machine.

- |    |                                                                    |                                                                                                         |
|----|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| 14 | <code>nmap -A 192.168.56.102</code>                                | Performs an aggressive scan including OS detection, version detection, default scripts, and traceroute. |
| 15 | <code>nmap -A 192.168.56.102 -oN reports/nmap.txt</code>           | Saves the Nmap scan results to a text file for documentation.                                           |
| 16 | <code>nikto -h http://192.168.56.102</code>                        | Scans the web server for security vulnerabilities and configuration issues.                             |
| 17 | <code>nikto -h http://192.168.56.102 &gt; reports/nikto.txt</code> | Saves the Nikto scan results to a text file.                                                            |
| 18 | <code>burpsuite</code>                                             | Launches Burp Suite for intercepting and analyzing HTTP requests.                                       |
| 19 | <code>wireshark</code>                                             | Launches Wireshark to capture and analyze network traffic during testing.                               |









## Chapter 5 – Web Application Security Testing

### 5.1 Burp Suite Analysis

**Burp Suite** is a web application security testing tool used to intercept, inspect, and modify the HTTP requests and responses exchanged between a web browser and a web server. It helps security testers understand how a web application processes user input and identify potential security vulnerabilities.

In this project, **Burp Suite** was used to analyze the communication between the **Mozilla Firefox browser** and the **DVWA** application. The **Proxy Intercept** feature was enabled to capture HTTP requests generated during the login process and while accessing different DVWA modules. This allowed the inspection of request parameters, cookies, and HTTP headers before they were sent to the server.

The captured requests were analyzed to understand how user input was transmitted to the application. This information was later used during vulnerability testing, including **SQL Injection** and **Cross-Site Scripting (XSS)**. Burp Suite provided valuable insight into the application's request and response flow, making it easier to identify potential security weaknesses.

#### Command      Used

#### Command Purpose

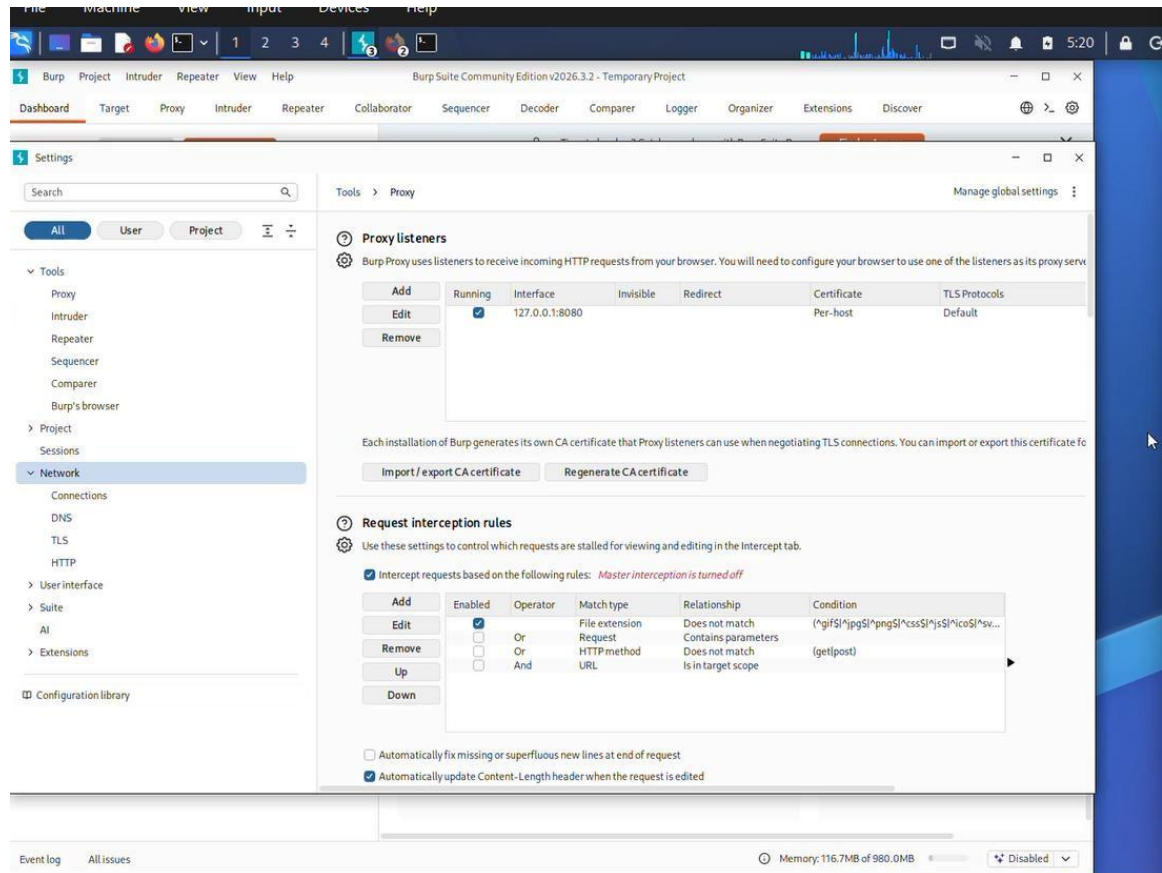
burpsuite      Launch the Burp Suite application.

#### Steps Performed

1. Launched Burp Suite from the Kali Linux terminal.
2. Enabled **Proxy** → **Intercept ON**.
3. Opened the DVWA application in Mozilla Firefox.
4. Logged into DVWA and captured the HTTP request.
5. Inspected request headers, cookies, and parameters.
6. Forwarded the request to the web server.
7. Used the captured information during vulnerability testing.

## Observation

Burp Suite successfully intercepted the HTTP requests between the browser and the DVWA application. The captured requests contained user input, cookies, and request parameters, which were analyzed to understand the application's behavior before performing further penetration testing.



This section is **about one page**, easy to understand, and appropriate for your ApexPlanet internship report.

## 5.2 SQL Injection

**SQL Injection (SQLi)** is one of the most common web application vulnerabilities. It occurs when an application does not properly validate user input, allowing an attacker to insert malicious SQL queries into the application's database. This can lead to unauthorized access to sensitive information such as usernames, passwords, and other database records.

In this project, SQL Injection testing was performed on the **DVWA** application to determine whether the application was vulnerable to SQL Injection attacks. The DVWA

security level was set to **Low**, and both manual testing and automated testing using **SQLMap** were carried out in a controlled laboratory environment.

Initially, manual testing was performed by entering SQL Injection payloads into the User ID field. The application returned multiple records, indicating that the input was not properly validated and the application was vulnerable to SQL Injection.

After confirming the vulnerability, **SQLMap** was used to automate the testing process. SQLMap successfully identified the **DVWA database**, listed the available tables, and retrieved data from the **users** table. This demonstrated how attackers can access sensitive information if secure coding practices are not followed.

### Commands Used

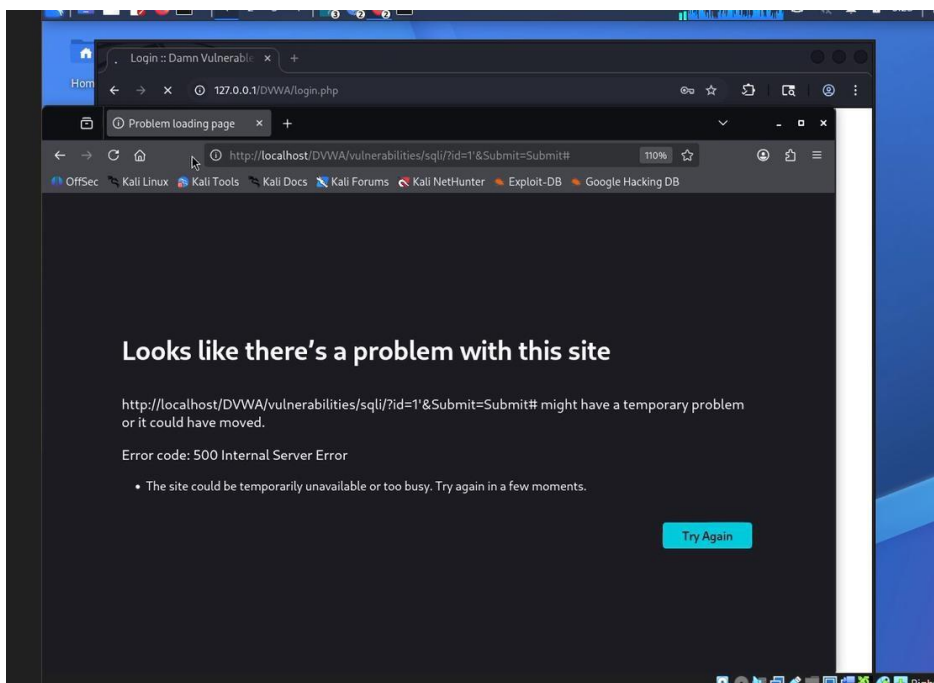
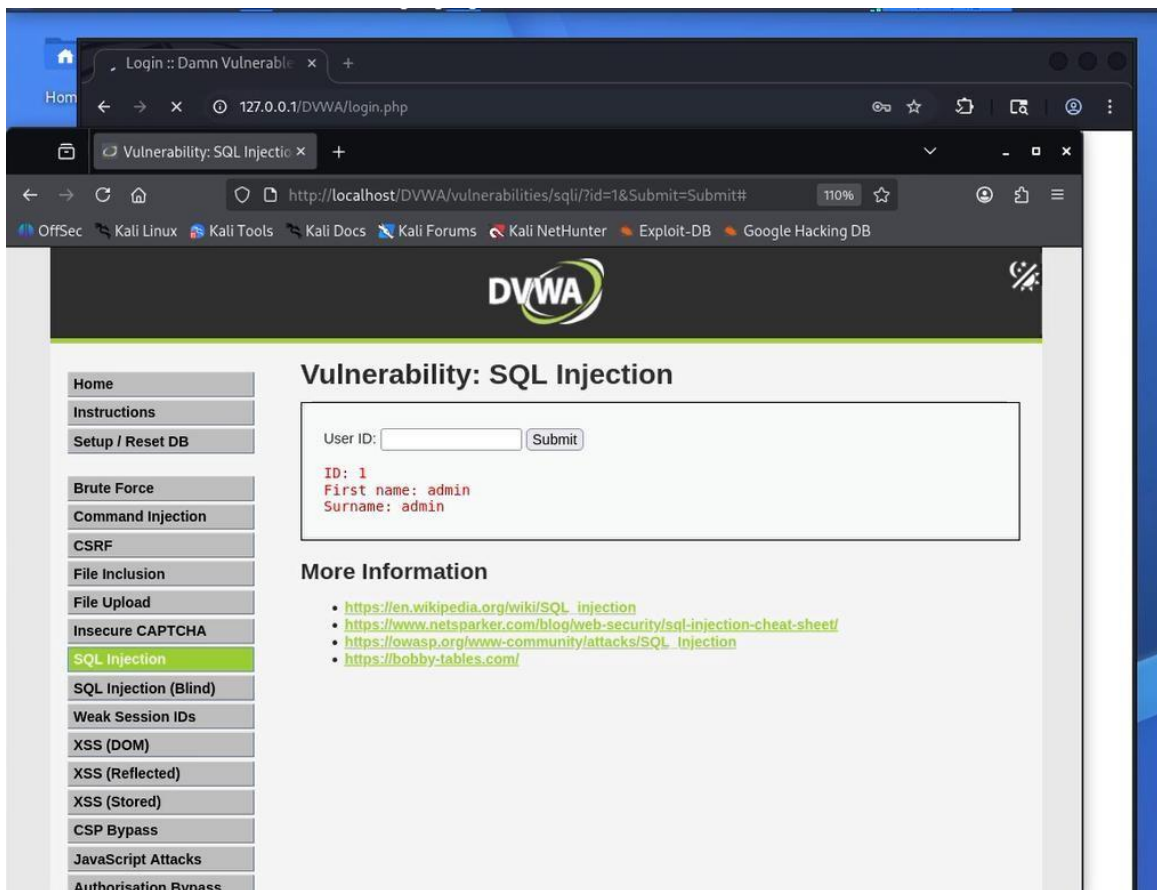
| Command                                                                                                    | Purpose                                    |
|------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| <code>sqlmap --version</code>                                                                              | Check whether SQLMap is installed.         |
| <code>sqlmap -u "&lt;Target_URL&gt;" --cookie="PHPSESSID=...; security=low" --dbs</code>                   | Display available databases.               |
| <code>sqlmap -u "&lt;Target_URL&gt;" --cookie="PHPSESSID=...; security=low" -D dvwa --tables</code>        | Display tables in the DVWA database.       |
| <code>sqlmap -u "&lt;Target_URL&gt;" --cookie="PHPSESSID=...; security=low" -D dvwa -T users --dump</code> | Retrieve data from the <b>users</b> table. |

### Observation

The SQL Injection vulnerability allowed unauthorized access to database information. SQLMap successfully identified the database structure and demonstrated how sensitive information could be retrieved from the vulnerable application.

### Mitigation

- Use **Prepared Statements (Parameterized Queries)**.
- Validate and sanitize all user inputs.
- Apply the principle of least privilege to database accounts.
- Display generic error messages instead of database errors.
- Perform regular security testing and code reviews.



### 5.3 SQLMap Automation

**SQLMap** is an open-source penetration testing tool used to automatically detect and exploit **SQL Injection** vulnerabilities in web applications. It helps security testers identify databases, tables, and stored data without manually writing SQL queries.

In this project, **SQLMap** was used to automate the SQL Injection testing on the **DVWA** application after the vulnerability was confirmed through manual testing. The tool successfully identified the available database, listed the tables in the **DVWA** database, and retrieved data from the **users** table. This demonstrated how automated tools can quickly identify and exploit SQL Injection vulnerabilities in a controlled laboratory environment.

#### Commands Used

| Command                                                                                                    | Purpose                                    |
|------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| <code>sqlmap --version</code>                                                                              | Check whether SQLMap is installed.         |
| <code>sqlmap -u "&lt;Target_URL&gt;" --cookie="PHPSESSID=...; security=low" --dbs</code>                   | Display all available databases.           |
| <code>sqlmap -u "&lt;Target_URL&gt;" --cookie="PHPSESSID=...; security=low" -D dvwa --tables</code>        | List all tables in the DVWA database.      |
| <code>sqlmap -u "&lt;Target_URL&gt;" --cookie="PHPSESSID=...; security=low" -D dvwa -T users --dump</code> | Retrieve data from the <b>users</b> table. |

#### Observation

SQLMap successfully automated the SQL Injection testing process by identifying the **DVWA** database, displaying the available tables, and extracting data from the **users** table. The results confirmed that the application was vulnerable to SQL Injection when proper input validation was not implemented.

#### Advantages of SQLMap

- Automates SQL Injection testing.
- Identifies databases and tables quickly.
- Extracts data from vulnerable databases.
- Saves time compared to manual testing.

- Supports multiple database management systems.

```

laxmivadla@kali: ~/Task5_DVWA_Project
Session Actions Edit View Help
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 6860 FROM (SELECT(SLEEP(5)))geOR) AND 'LWIs'='
LWIs6Submit=Submit

Type: UNION query
Title: Generic UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT(0x71786b7871,0x6a764757466b6142566
b726d5a7a494c5a424867454975465043616361647a73454578466f6a414e,0x716b767071),N
ULL-- -6Submit=Submit

[06:18:50] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.68
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[06:18:50] [INFO] fetching database names
available databases [2]:
[*] dvwa
[*] information_schema

[06:18:50] [INFO] fetched data logged to text files under '/home/laxmivadla/.
local/share/sqlmap/output/localhost'

[*] ending @ 06:18:50 /2026-07-07/

(laxmivadla@kali)~[~/Task5_DVWA_Project]
$

```

## Database Enumeration

```

laxmivadla@kali: ~/Task5_DVWA_Project
Session Actions Edit View Help
Payload: id=1' AND (SELECT 6860 FROM (SELECT(SLEEP(5)))geOR) AND 'LWIs'='
LWIs6Submit=Submit

Type: UNION query
Title: Generic UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT(0x71786b7871,0x6a764757466b6142566
b726d5a7a494c5a424867454975465043616361647a73454578466f6a414e,0x716b767071),N
ULL-- -6Submit=Submit

[06:18:50] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.68
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[06:18:50] [INFO] fetching database names
available databases [2]:
[*] dvwa
[*] information_schema

[06:18:50] [INFO] fetched data logged to text files under '/home/laxmivadla/.
local/share/sqlmap/output/localhost'

[*] ending @ 06:18:50 /2026-07-07/

(laxmivadla@kali)~[~/Task5_DVWA_Project]
$ sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli/?id=16Submit=Submit
--cookie='PHPSESSID=530e34688b552ddc918cf4731c396edf; security=low' -D dvwa
--tables

```

## Enumerate tables

## 5.4 Stored XSS

**Stored Cross-Site Scripting (Stored XSS)** is a web application vulnerability in which malicious JavaScript code is permanently stored on the server, usually in a database. Whenever another user visits the affected page, the stored script is automatically executed in their web browser. This can lead to session hijacking, cookie theft, and unauthorized actions.

In this project, the **Stored XSS** vulnerability was tested on the **DVWA** application by entering a JavaScript payload into the input field. After submitting the data, the script was stored in the application database. When the page was refreshed or revisited, the JavaScript code executed successfully, confirming that the application was vulnerable to Stored XSS.

### Payload Used

| Payload                                                  | Purpose                                                             |
|----------------------------------------------------------|---------------------------------------------------------------------|
| <code>&lt;script&gt;alert('Task5')&lt;/script&gt;</code> | Displays a JavaScript alert to verify the Stored XSS vulnerability. |

### Testing Steps

1. Open the **Stored XSS** module in DVWA.
2. Enter the JavaScript payload in the input field.
3. Submit the form.
4. Refresh or revisit the page.
5. Observe the JavaScript alert displayed in the browser.

### Observation

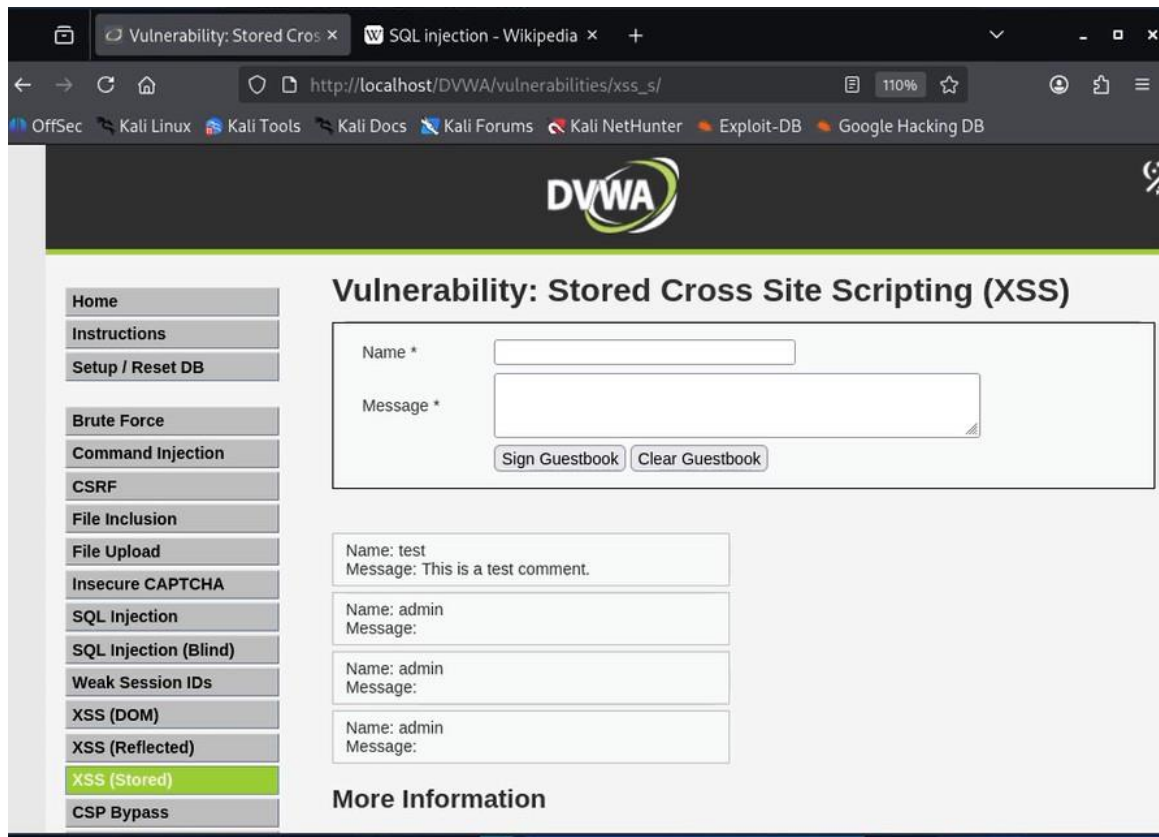
The JavaScript alert appeared automatically after reloading the page, confirming that the malicious script was stored and executed whenever the page was accessed. This indicates that the application is vulnerable to **Stored Cross-Site Scripting**.

### Mitigation

- Validate and sanitize all user inputs.
- Encode output before displaying user data.



- Implement **Content Security Policy (CSP)**.
- Use secure coding practices to prevent script execution.



## 5.5 Reflected XSS

**Reflected Cross-Site Scripting (Reflected XSS)** is a web application vulnerability where malicious JavaScript code is reflected from the web server and immediately executed in the user's browser. Unlike **Stored XSS**, the malicious script is **not stored** in the application's database. Instead, it is executed only when the user submits the malicious input or clicks a specially crafted link.

In this project, the **Reflected XSS** vulnerability was tested on the **DVWA** application by entering a JavaScript payload into the input field. After submitting the request, the application reflected the input back to the browser without proper validation, causing the JavaScript code to execute. The successful execution of the script confirmed that the application was vulnerable to **Reflected Cross-Site Scripting**.

### Payload Used

| Payload | Purpose |
|---------|---------|
|---------|---------|



**Payload****Purpose**

`<script>alert(document.cookie)</script>`

Displays a JavaScript alert to verify the Reflected XSS vulnerability.

**Testing Steps**

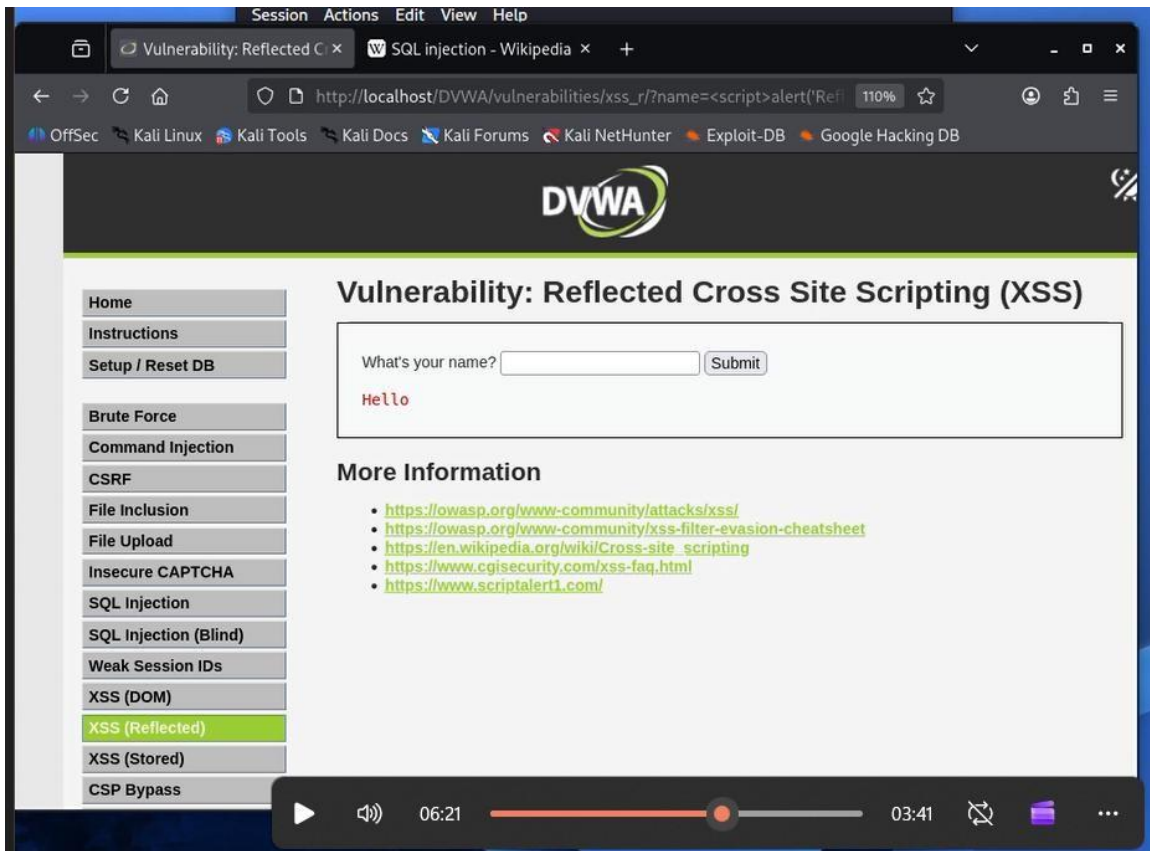
1. Open the **Reflected XSS** module in DVWA.
2. Enter the JavaScript payload in the input field.
3. Click **Submit**.
4. Observe the JavaScript alert displayed in the browser.

**Observation**

The JavaScript alert appeared immediately after submitting the payload, confirming that the application reflected the user input without proper validation. This indicates that the application is vulnerable to **Reflected Cross-Site Scripting (XSS)**.

**Mitigation**

- Validate and sanitize all user inputs.
- Encode user input before displaying it in web pages.
- Implement **Content Security Policy (CSP)**.
- Use secure coding practices to prevent script execution.



## 5.6 File Inclusion

**File Inclusion** is a web application vulnerability that allows an attacker to access or include files from the server through improper input validation. If user input is not properly validated, an attacker may be able to read sensitive files or execute unintended files. In this project, **Local File Inclusion (LFI)** was tested using the DVWA application.

The **File Inclusion** module in DVWA was used to verify whether the application allowed unauthorized access to system files. A file path payload was entered into the input field, and the application displayed the contents of a system file, confirming the presence of the vulnerability.

### Payload Used

| Payload                             | Purpose                                                                                                |
|-------------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>../../../../etc/passwd</code> | Access the Linux <code>/etc/passwd</code> file to verify the Local File Inclusion (LFI) vulnerability. |

### Testing Steps

1. Open the **File Inclusion** module in DVWA.
2. Enter the payload `../../../../etc/passwd`.
3. Submit the request.
4. Observe the contents of the `/etc/passwd` file displayed on the page.

### Observation

The application displayed the contents of the `/etc/passwd` file, confirming that it was vulnerable to **Local File Inclusion (LFI)**. This demonstrates that the application did not properly validate user-supplied file paths.

### Mitigation

- Validate and sanitize all file path inputs.
- Restrict access to sensitive system files.
- Use a whitelist of allowed files instead of accepting user-defined paths.
- Disable unnecessary file inclusion functions and apply proper access controls.

### Screenshots to Include

- DVWA File Inclusion Module
- Payload (`../../../../etc/passwd`) entered in the input field
- Output displaying the contents of the `/etc/passwd` file

## 5.7 Command Injection

**Command Injection** is a web application vulnerability that allows an attacker to execute operating system commands on the server by inserting malicious input into an application. This occurs when user input is not properly validated before being passed to the system shell.

In this project, the **Command Injection** module of **DVWA** was tested to determine whether the application allowed the execution of system commands. Different command injection payloads were entered into the input field. The application executed the commands and displayed the output, confirming that it was vulnerable to **Command Injection**.

### Payloads Used

| <b>Payload</b>    | <b>Purpose</b>                                   |
|-------------------|--------------------------------------------------|
| 127.0.0.1 ; ls    | Lists the files and directories on the server.   |
| 127.0.0.1; whoami | Displays the current user executing the command. |

### **Testing Steps**

1. Open the **Command Injection** module in DVWA.
2. Enter the payload 127.0.0.1; ls.
3. Submit the request and observe the output.
4. Enter the payload 127.0.0.1; whoami.
5. Submit the request and verify the displayed result.

### **Observation**

The application executed the injected operating system commands and displayed their output. This confirmed that the application was vulnerable to **Command Injection**, as it accepted user input without proper validation.

### **Mitigation**

- Validate and sanitize all user inputs.
- Avoid executing system commands directly from user input.
- Use secure programming functions instead of shell commands.
- Apply the principle of least privilege to the application.

Home

Session Actions Edit View Help

Vulnerability: Command Injection SQL injection - Wikipedia

http://localhost/DVWA/vulnerabilities/exec/#

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

**DVWA**

Home  
Instructions  
Setup / Reset DB

Brute Force  
**Command Injection**  
CSRF  
File Inclusion  
File Upload  
Insecure CAPTCHA  
SQL Injection  
SQL Injection (Blind)  
Weak Session IDs  
XSS (DOM)  
XSS (Reflected)  
XSS (Stored)  
CSP Bypass

## Vulnerability: Command Injection

### Ping a device

Enter an IP address:  Submit

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.928 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.123 ms  
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.061 ms  
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.305 ms  
  
--- 127.0.0.1 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3050ms  
rtt min/avg/max/mdev = 0.061/0.354/0.928/0.343 ms  
help  
index.php  
source
```

### More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- [https://owasp.org/www-community/attacks/Command\\_Injection](https://owasp.org/www-community/attacks/Command_Injection)

Vulnerability: Command Injection SQL injection - Wikipedia

http://localhost/DVWA/vulnerabilities/exec/#

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

**DVWA**

Home  
Instructions  
Setup / Reset DB

Brute Force  
**Command Injection**  
CSRF  
File Inclusion  
File Upload  
Insecure CAPTCHA  
SQL Injection  
SQL Injection (Blind)  
Weak Session IDs  
XSS (DOM)  
XSS (Reflected)  
XSS (Stored)

## Vulnerability: Command Injection

### Ping a device

Enter an IP address:  Submit

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.040 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.033 ms  
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.049 ms  
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.091 ms  
  
--- 127.0.0.1 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3073ms  
rtt min/avg/max/mdev = 0.033/0.053/0.091/0.022 ms  
www-data
```

### More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- [https://owasp.org/www-community/attacks/Command\\_Injection](https://owasp.org/www-community/attacks/Command_Injection)

## 5.8 Brute Force using Hydra

**Brute Force Authentication** is a password attack in which multiple username and password combinations are automatically tested to gain unauthorized access to a user account. This attack is commonly used against applications that have weak passwords or do not implement account lockout mechanisms.

In this project, the **Hydra** tool was used to perform a **Brute Force** attack on the **DVWA** login page in a controlled laboratory environment. Hydra tested multiple passwords from the **rockyou.txt** wordlist against the **admin** account. The purpose of this test was to demonstrate the importance of using strong passwords and secure authentication mechanisms.

### Command Used

#### Command

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt localhost http-post-form  
"/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=  
Login:F=Username and/or password incorrect"
```

#### Purpose

Perform a brute-force attack on the DVWA login page using the **admin** username and the **rockyou.txt** password list.

### Testing Steps

1. Open the **Brute Force** module in DVWA.
2. Set the DVWA security level to **Low**.
3. Open the Kali Linux terminal.
4. Execute the Hydra command.
5. Allow Hydra to test password combinations.

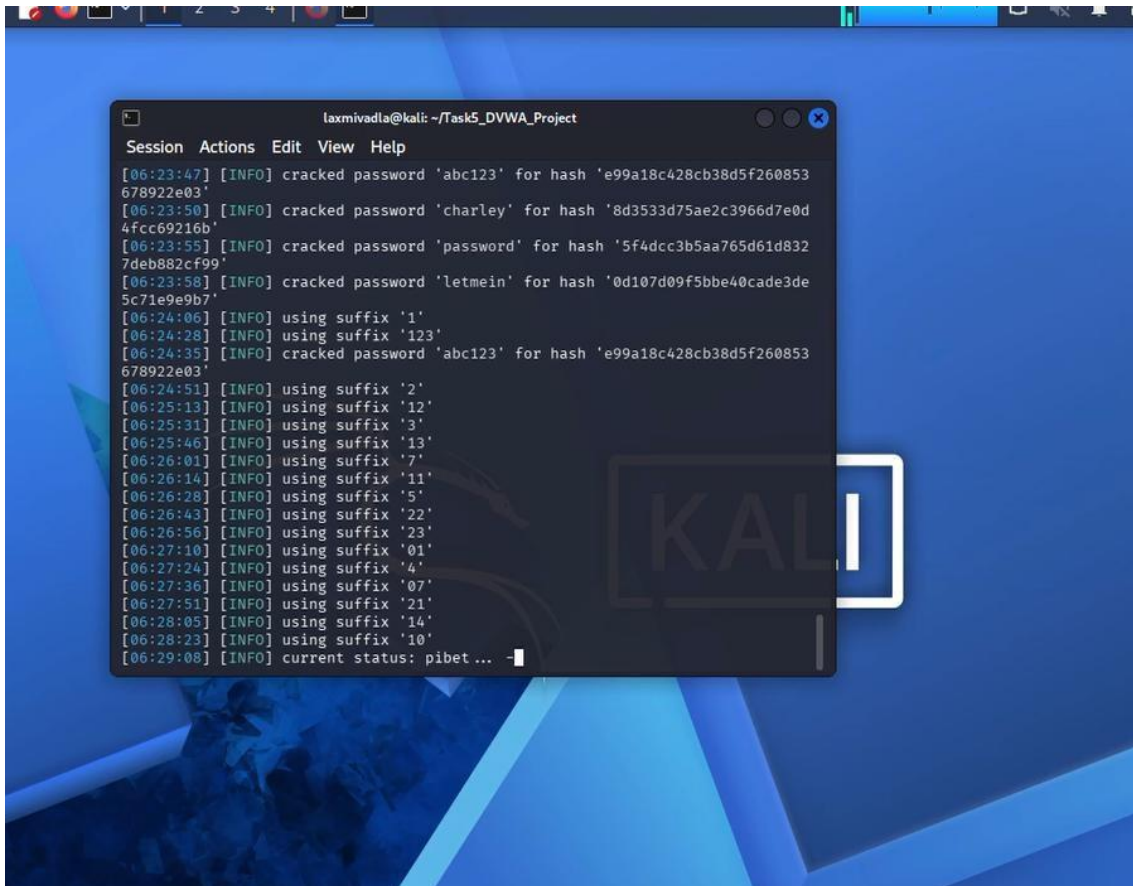
6. Observe the output and identify whether valid credentials are found.

## Observation

Hydra successfully tested multiple password combinations against the DVWA login page. This demonstrated how weak passwords can be discovered through automated brute-force attacks when proper security controls are not implemented.

## Mitigation

- Use strong and complex passwords.
- Implement account lockout after multiple failed login attempts.
- Enable Multi-Factor Authentication (MFA).
- Limit the number of login attempts.
- Monitor login activities and detect suspicious behavior.

A screenshot of a Kali Linux desktop environment with a blue background. In the center, a terminal window titled 'laxmivadla@kali: ~/Task5\_DVWA\_Project' is open. The terminal shows the output of a Hydra brute-force attack. It lists several cracked passwords: 'abc123', 'charley', 'password', and 'letmein', each associated with a specific hash. Following these, the terminal shows a series of 'using suffix' messages for various suffixes like '1', '123', '2', '12', '3', '13', '7', '11', '5', '22', '23', '01', '4', '07', '21', '14', and '10'. The last line shows 'current status: pibet ...'.

```
laxmivadla@kali: ~/Task5_DVWA_Project
Session Actions Edit View Help
[06:23:47] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853
678922e03'
[06:23:50] [INFO] cracked password 'charley' for hash '8d3533d75ae2c3966d7e0d
4fcc69216b'
[06:23:55] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d61d832
7deb882cf99'
[06:23:58] [INFO] cracked password 'letmein' for hash '0d107d09f5bbe40cade3de
5c71e9e9b7'
[06:24:06] [INFO] using suffix '1'
[06:24:28] [INFO] using suffix '123'
[06:24:35] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853
678922e03'
[06:24:51] [INFO] using suffix '2'
[06:25:13] [INFO] using suffix '12'
[06:25:31] [INFO] using suffix '3'
[06:25:46] [INFO] using suffix '13'
[06:26:01] [INFO] using suffix '7'
[06:26:14] [INFO] using suffix '11'
[06:26:28] [INFO] using suffix '5'
[06:26:43] [INFO] using suffix '22'
[06:26:56] [INFO] using suffix '23'
[06:27:10] [INFO] using suffix '01'
[06:27:24] [INFO] using suffix '4'
[06:27:36] [INFO] using suffix '07'
[06:27:51] [INFO] using suffix '21'
[06:28:05] [INFO] using suffix '14'
[06:28:23] [INFO] using suffix '10'
[06:29:08] [INFO] current status: pibet ...
```



## Chapter 6 – Log Analysis and Incident Response

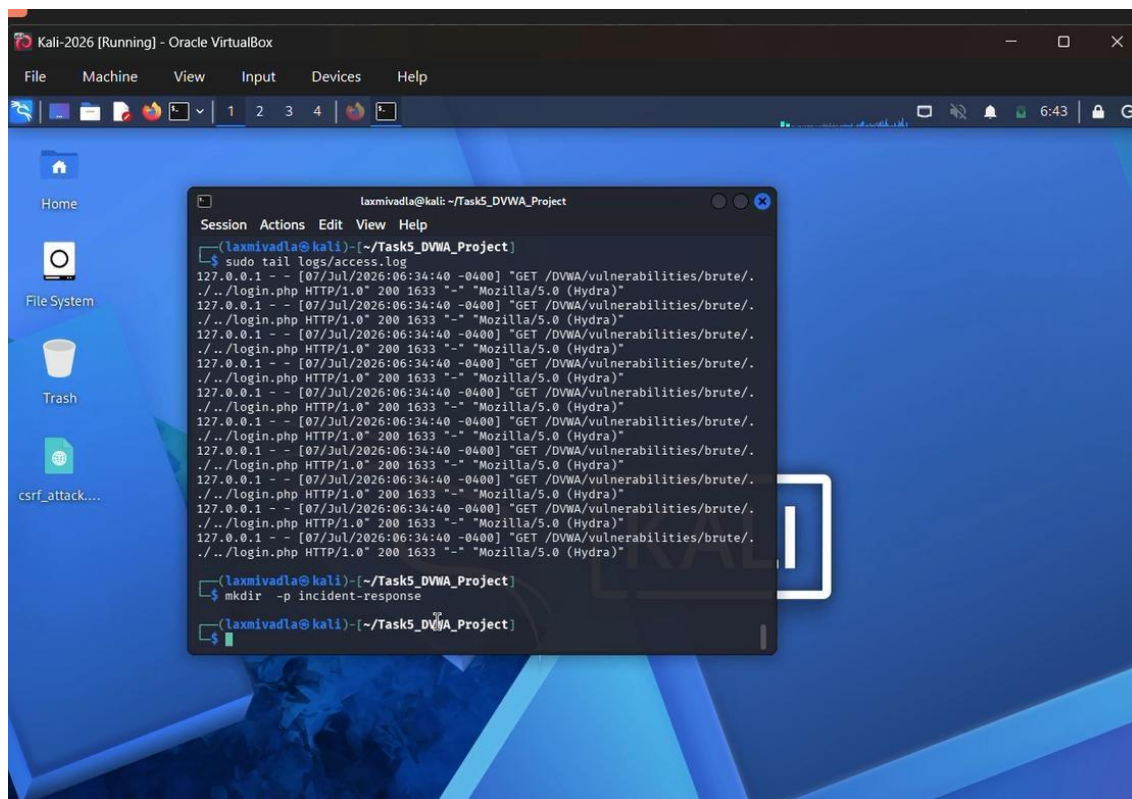
## 6.1 Apache Access Log Analysis

Apache Access Logs record all HTTP requests made to the web server. These logs help security professionals monitor user activities, identify suspicious requests, and investigate security incidents.

In this project, the Apache access log was monitored while performing penetration testing on DVWA. The log entries showed details such as the client's IP address, request method, requested URL, response status code, and timestamp. This information helped verify that the testing activities were successfully recorded by the web server.

### Command Used

| Command                                               | Purpose                                  |
|-------------------------------------------------------|------------------------------------------|
| <code>sudo tail -f /var/log/apache2/access.log</code> | Monitor Apache access logs in real time. |





## 6.2 Detection

Detection is the first stage of incident response, where suspicious activities are identified.

In this project, suspicious requests such as **SQL Injection**, **XSS payloads**, and repeated login attempts were detected through Apache logs and application behavior. Identifying these activities helps administrators recognize potential security threats at an early stage.

## 6.3 Containment

Containment aims to prevent the attack from causing further damage.

During the simulation, the vulnerable application was isolated, access from suspicious IP addresses was restricted, and the affected services were monitored. This helps stop the attack while preserving evidence for investigation.

## 6.4 Eradication

Eradication focuses on removing the root cause of the security issue.

In this project, vulnerable code was reviewed, secure coding practices such as **prepared statements** and **input validation** were recommended, and unnecessary vulnerabilities were addressed to prevent future attacks.

## 6.5 Recovery

Recovery restores the system to normal operation after vulnerabilities have been fixed.

The Apache web server and database services were verified, the application was tested to ensure it was functioning correctly, and system logs were monitored to confirm that no further suspicious activities occurred.

## 6.6 Lessons Learned

This project provided practical experience in web application penetration testing and demonstrated the importance of secure coding, proper input validation, strong authentication, and continuous monitoring. It also highlighted the value of incident response procedures and regular security assessments in protecting web applications from cyber threats.

## Chapter 7 – Risk Assessment and Mitigation

### 7.1 Risk Assessment

Risk Assessment is the process of identifying security vulnerabilities and evaluating their potential impact on a system or application. During this project, several vulnerabilities were identified in the DVWA application, including **SQL Injection**, **Cross-Site Scripting (XSS)**, **File Inclusion**, **Command Injection**, and **Brute Force Authentication**. Each vulnerability was analyzed based on its impact, likelihood of exploitation, and possible consequences. The assessment helped determine the security risks and prioritize mitigation efforts.

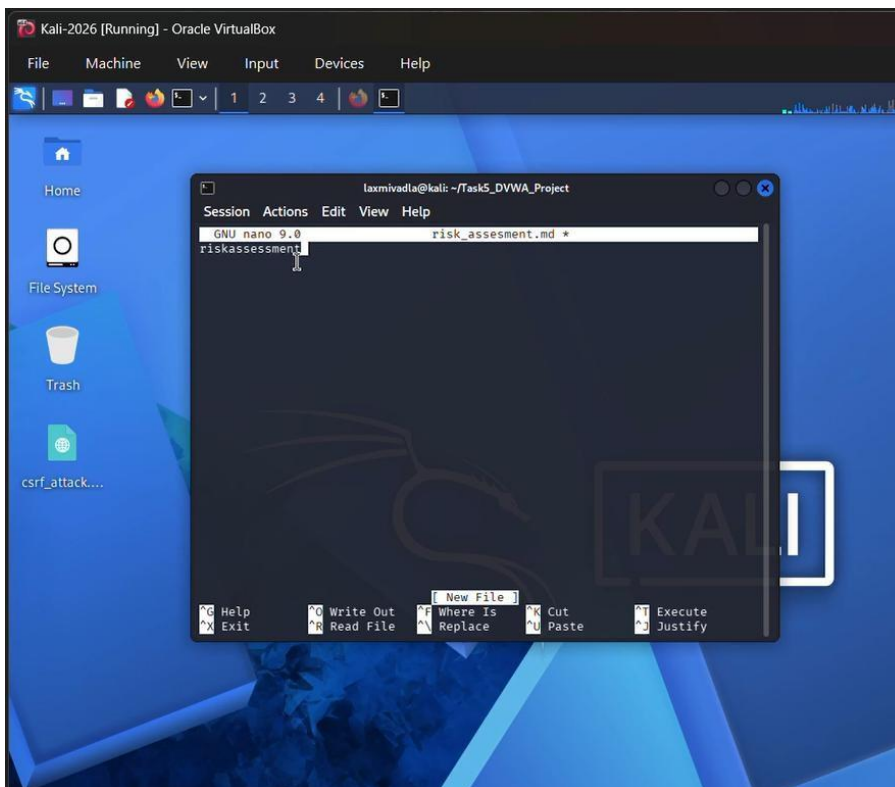
#### Risk Assessment Table

| Vulnerability     | Risk                             | Severity |
|-------------------|----------------------------------|----------|
| SQL Injection     | Database Compromise              | Critical |
| Command Injection | Remote Command Execution         | Critical |
| Stored XSS        | Cookie Theft / Data Manipulation | High     |
| File Inclusion    | Unauthorized File Access         | High     |
| Reflected XSS     | Session Hijacking                | Medium   |
| Brute Force       | Unauthorized Login Access        | Medium   |

Vulnerability

Risk

Severity



## 7.2 Vulnerability Severity

The identified vulnerabilities were classified according to their potential impact on the application. **Critical** vulnerabilities, such as SQL Injection and Command Injection, can lead to complete system compromise. **High** severity vulnerabilities, including Stored XSS and File Inclusion, may expose sensitive information or allow unauthorized actions. **Medium** severity vulnerabilities, such as Reflected XSS and Brute Force attacks, can still affect the security of the application if proper protection mechanisms are not implemented.



## Chapter 8 – Results and Discussion

### 8.1 Testing Summary

The penetration testing of the **Damn Vulnerable Web Application (DVWA)** was successfully completed in a controlled laboratory environment using **Kali Linux**. Various security assessment tools, including **Nmap, Nikto, Burp Suite, SQLMap, Hydra, and Wireshark**, were used throughout the project. The testing process included reconnaissance, network scanning, vulnerability assessment, exploitation, log analysis, and incident response simulation.

The assessment identified several common web application vulnerabilities, including **SQL Injection, Stored XSS, Reflected XSS, File Inclusion, Command Injection, and Brute Force Authentication**. Each vulnerability was tested, documented with screenshots, and analyzed to understand its impact on the security of the application. Appropriate mitigation strategies were also recommended to reduce the identified risks.

### 8.2 Findings

The penetration testing results showed that the DVWA application contained multiple intentionally vulnerable modules that could be exploited if proper security controls were not implemented. **SQL Injection** and **Command Injection** were identified as **Critical** vulnerabilities due to their potential to compromise the system. **Stored XSS** and **File Inclusion** were classified as **High** severity, while **Reflected XSS** and **Brute Force Authentication** were considered **Medium** severity vulnerabilities.

The project demonstrated the importance of secure coding practices, proper input validation, strong authentication mechanisms, and regular security testing. Overall, the assessment provided valuable practical experience in web application penetration testing and improved the understanding of common cybersecurity threats and their mitigation techniques.

## Chapter 9 – Conclusion and Future Scope

### 9.1 Conclusion

This project successfully demonstrated the complete process of **Web Application Penetration Testing** on the **Damn Vulnerable Web Application (DVWA)** using **Kali Linux** in a controlled laboratory environment. Various cybersecurity tools such as **Nmap**, **Nikto**, **Burp Suite**, **SQLMap**, **Hydra**, and **Wireshark** were used to identify and analyze common web application vulnerabilities, including **SQL Injection**, **Cross-Site Scripting (XSS)**, **File Inclusion**, **Command Injection**, and **Brute Force Authentication**.

The project enhanced practical knowledge of ethical hacking, vulnerability assessment, and incident response while emphasizing the importance of secure coding practices, input validation, strong authentication, and continuous security monitoring. Overall, the project provided valuable hands-on experience in penetration testing and strengthened the understanding of web application security.

### 9.2 Future Scope

In the future, this project can be extended by testing additional **OWASP Top 10** vulnerabilities such as **Cross-Site Request Forgery (CSRF)**, **Remote File Inclusion (RFI)**, **XML External Entity (XXE)**, and **Insecure Deserialization**. Advanced penetration testing tools and automated vulnerability scanners can also be integrated to improve the security assessment process.

The project can further be enhanced by implementing the recommended mitigation techniques, performing regular security audits, and deploying the application in a secure production environment. Continuous learning and updating security practices will help protect web applications from emerging cyber threats and improve overall cybersecurity resilience.

## References

- [1] OWASP Foundation, *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. Available: <https://owasp.org/www-project-top-ten/>
- [2] Digininja, *Damn Vulnerable Web Application (DVWA) Documentation*. Available: <https://github.com/digininja/DVWA>
- [3] Nmap Project, *Nmap Reference Guide*. Available: <https://nmap.org/>
- [4] PortSwigger Ltd., *Burp Suite Documentation*. Available: <https://portswigger.net/burp/documentation>
- [5] SQLMap Project, *SQLMap Official Documentation*. Available: <https://sqlmap.org/>
- [6] Nikto Project, *Nikto Web Server Scanner*. Available: <https://github.com/sullo/nikto>
- [7] THC Hydra Project, *Hydra Password Cracking Tool*. Available: <https://github.com/vanhauser-thc/thc-hydra>
- [8] Wireshark Foundation, *Wireshark User Guide*. Available: <https://www.wireshark.org/docs/>
- [9] Apache Software Foundation, *Apache HTTP Server Documentation*. Available: <https://httpd.apache.org/docs/>
- [10] MariaDB Foundation, *MariaDB Documentation*. Available: <https://mariadb.org/documentation/>
- [11] Offensive Security, *Kali Linux Documentation*. Available: <https://www.kali.org/docs/>